

First Gear Bank v1 Foundation Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Build the verified, test-driven foundation for First Gear Bank: a safe package pipeline, engine-independent banking core, fixed-point domain model, balanced journal, atomic coordinator, sectioned persistence, and a thin Vintage Story 1.22.7 world-save host.

Architecture: Keep all financial rules and deterministic state transitions in `FirstGearBank.Core`, which references no Vintage Story assembly. The existing `First Gear Bank` project remains the packaged engine adapter and translates verified 1.22.7 lifecycle calls into narrow core ports. State is published as immutable revisions and saved as independently checksummed sections so corruption can isolate a subsystem without inventing or silently resetting money.

Tech Stack: .NET SDK 10.0.400, C# 14, Vintage Story API 1.22.7, xUnit.net v3 4.0.0 with Microsoft Testing Platform v2, Cake Frosting 6.2.0, `System.Text.Json`, `System.IO.Compression`, and SHA-256/HMAC-SHA-256 from the base class library.

Spec: docs/superpowers/specs/2026-09-08-first-gear-bank-v1-design.md

Global Constraints

- Target Vintage Story exactly 1.22.7 and use mod ID exactly `firstgearbank`.
- If execution uses a Git worktree, create it as a direct sibling of this repository under the `mods` directory so the required default reference path `../Vintage Story Reference` remains valid.
- Ship one mod package containing a pure .NET 10 core plus a thin Vintage Story adapter.
- Keep all financial authority on the server; the client never supplies a trusted UID, balance, timestamp, rate, or result.
- Use six-decimal signed fixed-point money: one gear is 1,000,000 bank units.
- Round every floating-rate result to the nearest bank unit with midpoint ties to even.
- Cap customer rusty-savings and temporal-vault balances at 2,147,483,647,000,000 units.
- Never allow negative savings or temporal-vault balances; signed system and future contract accounts remain valid.
- Treat the append-only, independently balanced journal as the monetary source of truth.
- Publish each command as one validated immutable revision containing zero or more complete journal records.
- Never treat default record equality over `ImmutableArray`, `ImmutableDictionary`, or `ReadOnlyMemory` as semantic equality; use explicit structural comparison, canonical bytes, or a domain validator.
- Preserve corrupt authoritative bytes and quarantine the affected domain; never synthesize a repair to money.
- Never display or normally log Vintage Story PlayerUIDs.
- Add no Harmony patch unless a public-API gap is documented and the user separately approves the exact patch.
- Keep Better Loot, Better Loot Plus, Config Lib, and Integrated Mod Manager optional.
- Do not implement any section 2.1 non-goal in this phase.
- Follow all checked-in common, C#, hygiene, and Vintage Story standards, including file headers, callable comments, exact three-blank-line spacing, 120-character documentation lines, and `INFO/DEBUG/WARN/CRIT` log labels.
- Use test-driven development for every behavior change: observe the intended failure, make the smallest passing change, then refactor while green.
- Run formatter verification, build, target-project tests, the full test suite, and package verification before a task is complete.

Phase Boundary

This plan intentionally stops before playable deposits, interest, CDs, transfers, GUI, or branches. Its output is a loadable foundation-only mod whose state and package contracts are safe enough for the remaining plans in `2026-09-08-first-gear-bank-v1-roadmap.md`.

Two proof gates occur here because they affect persisted contracts:

1. The physical-settlement spike records whether player mod data is provably co-serialized with inventory. If it is not, the supported recovery policy is authenticated evidence where available and quarantine for every ambiguous state.
2. The accrued-liability-index spike must prove a sublinear exact result for the specification's per-account, month-by-month ties-to-even accrual. A scalar aggregate is known to be insufficient. If the proof fails, stop before the CD/liquidity projection schema is frozen and present the user with the measured conflict.

File and Responsibility Map

AGENTS.md	repository-specific agent entry point
README.md	setup, architecture, build, test, and package commands
Directory.Build.props	common C# compiler and reproducibility settings
Directory.Packages.props	centrally pinned third-party packages
global.json	.NET 10.0.400 and Microsoft Testing Platform selection
FirstGearBank.Core/ FirstGearBank.Core.csproj CoreAssemblyMarker.cs Domain/ Money/ Time/ Configuration/ Ledger/ Projection/ Requests/ Coordination/ Security/ Settlement/ Persistence/	engine-independent library dependency-boundary test anchor currencies, identifiers, results, revisions fixed-point arithmetic, parsing, formatting, caps normalized financial and audit timestamps JSONC parsing, defaults, validation, activation candidates typed immutable records, postings, hash chain, validation replay-built account and system views scope sequencing, payload digests, cached terminal results serialized command admission and immutable publication production entropy port/implementation recovery evidence contracts and authentication section framing, codecs, migrations, limits, quarantine
First Gear Bank/ FirstGearBankModSystem.cs Runtime/FirstGearBankServerHost.cs Persistence/VintageStoryWorldStateStore.cs Configuration/VintageStoryConfigurationStore.cs Diagnostics/FgbFileLogger.cs Diagnostics/VintageStoryBankLogger.cs	Vintage Story registration and lifecycle wiring only testable startup/load/save/dispose orchestration adapter for the namespaced install marker and state envelope comment-preserving config file ownership bounded file sink with exact severity labels file plus native Vintage Story log fan-out
CakeBuild/ BuildPaths.cs ModManifest.cs Packaging/ Tasks/	distinct repository/project/stage/release/deploy paths build-only manifest parsing with no game dependency deterministic ZIP construction, inspection, safe selection unique, nondestructive build/test/package/deploy graph
tests/ FirstGearBank.Core.Tests/ FirstGearBank.Adapter.Tests/ CakeBuild.Tests/	deterministic domain and state-machine tests host/config/log tests behind project-owned fakes packaging and deployment safety tests
docs/research/ docs/architecture/ docs/testing/	source-linked Vintage Story evidence recovery, identity/time, persistence, and index decisions controlled 1.22.7 engine smoke procedures and results

Task 1: Record the 1.22.7 Evidence and Foundation Decisions

Files:

- Create: docs/research/vintage-story-1.22.7-api-evidence.md
- Create: docs/architecture/0001-physical-settlement-recovery-threat-model.md
- Create: docs/architecture/0002-economically-accrued-liability-index.md
- Create: docs/architecture/0003-domain-identifiers-and-financial-time.md
- Create: docs/architecture/0004-persistence-sections-and-revisions.md
- Create: docs/testing/foundation-engine-smoke.md
- Create: same-basename .tex files for all six Markdown documents through Pandoc
- Generate but do not commit: same-basename .pdf files for rendered review

Interfaces:

- Consumes: the locked v03 specification and the indexed 1.22.7 reference at ../Vintage Story Reference.
- Produces: source-linked go/no-go evidence and the serialized conventions consumed by every following task.

☐ Step 1: Re-run narrow reference queries and capture the exact source paths

```
$db = "..\Vintage Story Reference\versions\1.22.7\Index\code.sqlite"
python -B "..\Vintage Story Reference\tools\code-index\query_code_index.py" --db $db --type ModSystem --limit 10
python -B "..\Vintage Story Reference\tools\code-index\query_code_index.py" --db $db --type ISaveGame --limit 10
python -B "..\Vintage Story Reference\tools\code-index\query_code_index.py" --db $db --type IServerEventAPI --limit 10
python -B "..\Vintage Story Reference\tools\code-index\query_code_index.py" --db $db --type IServerPlayer --limit 10
```

```
python -B "..\Vintage Story Reference\tools\code-index\query_code_index.py" --db $db --type INetworkAPI --limit 10
python -B "..\Vintage Story Reference\tools\code-index\query_code_index.py" --db $db --type ILogger --limit 10
```

Expected: every query resolves only 1.22.7 symbols; no 1.21 or earlier type is used as implementation authority.

□ Step 2: Write the API evidence matrix

Use this exact column contract and fill each row with a concrete source file and line span:

```
| Capability | Verified symbol or asset | 1.22.7 source | Conclusion | Dependent phase |
|---|---|---|---|
| Common/server lifecycle | `ModSystem.Start`, `StartServerSide`, `Dispose` | `API/VintagestoryAPI_decompiled_1.22.7/Vintagestory/API/Common/ModSystem.cs:74,112,119` | Verified public API | Foundation |
| World state | `ISaveGame.GetData`, `StoreData`, `SavegameIdentifier` | `API/VintagestoryAPI_decompiled_1.22.7/Vintagestory/API/Server/ISaveGame.cs:30,54,61` | Verified blob API; no transaction/fsync guarantee | Foundation |
| Save events | `SaveGameCreated`, `SaveGameLoaded`, `GameWorldSave` | `API/VintagestoryAPI_decompiled_1.22.7/Vintagestory/API/Server/IServerEventAPI.cs:124,129,142-144` | Verified pre-write staging event; no post-save acknowledgment | Foundation |
| Player capsule | `IServerPlayer.SetModData`, `GetModData` | `API/VintagestoryAPI_decompiled_1.22.7/Vintagestory/API/Server/IServerPlayer.cs:114,116` | API exists; co-serialization with inventory unproved | Settlement spike |
| Reusable recipe input | `CraftingRecipeIngredient.Consume`; vanilla glider/copy recipes | `API/VintagestoryAPI_decompiled_1.22.7/Vintagestory/API/Common/CraftingRecipeIngredient.cs:136`; `Assets/survival/recipes/grid/glider.json:8`; `schematiccopy.json:5` | Use `consume: false`, never `isTool` | Charter |
```

Also include verified rows for current Playing connections, typed networking, registrations, entity spawning, GUI, commands/permissions, the data path, rusty/temporal item codes, and asset layout. Mark player/inventory durability, Charter action interception, Banker base/dialogue, generated-trader identity, and absent optional mods as explicit gates.

□ Step 3: Record the physical-settlement threat model

The decision record must contain this trust statement verbatim:

The authenticated server and its world save are trusted. Clients, packet contents, item attributes supplied by clients, stale player snapshots, filenames, and unauthenticated recovery bytes are not trusted. A valid capsule proves only the canonical evidence it authenticates; it does not prove an unstated save-order guarantee. If the Bank record, recovery key, world binding, player binding, manifest, fingerprint, phase, or authentication tag cannot be reconciled, First Gear Bank quarantines the affected physical settlement and neither creates nor destroys compensating value by inference.

Specify HMAC-SHA-256, a random 256-bit world recovery key, binding to the hash of `ISaveGame.SavegameIdentifier`, canonical length-prefixed UTF-8/binary encoding, constant-time tag comparison, key nonrotation while evidence exists, and raw-evidence retention.

□ Step 4: Run and document the accrued-liability counterexample and index gate

Use this minimal counterexample in the decision record:

```
balances = [1, 1] bank unit
factor    = 1.5
sum(roundToEven(factor * each balance)) = 2 + 2 = 4
roundToEven(factor * sum(balances))    = roundToEven(3) = 3
```

The record must require an indexed algorithm to match the per-account month-by-month function for adversarial balances, partial months, caps, and account mutations. If no exact sublinear algorithm is proved, record Gate result: failed and stop before Phase 4 of the roadmap; do not substitute aggregate rounding.

□ Step 5: Record identifier, time, section, and revision encodings

GUID-backed IDs: 16 canonical bytes in binary; lowercase 32-hex characters in diagnostic text.
 Player key: exact authenticated engine UID internally; bounded UTF-8 string; never normalized or player-facing.
 Financial instant: nonnegative completed financial months plus decimal fractional month in [0, 1).
 Journal sequence: signed Int64 beginning at 1 and increasing contiguously.
 Global revision: signed Int64 beginning at 0; one increment for every published state revision.
 Subsystem revision: one signed Int64 per independently persisted domain; increment only when that domain changes.
 Hash domains: explicit ASCII prefix and version followed by canonical length-prefixed bytes; never `GetHashCode`.

□ Step 6: Write the controlled engine smoke matrix

The procedure must use a disposable 1.22.7 test world and record, for each run, game version, mod ZIP hash, world backup, action, expected state, observed state, FGB log excerpt, and pass/fail. Include fresh load, save/restart, missing blob, corrupt copied blob, player-mod-data persistence, and absence of all optional mods. List inventory-plus-capsule save ordering and server shutdown during every physical-settlement phase as explicitly deferred, nonblocking Phase 3 rows: Phase 1 has no inventory mutation capable of producing those phases and must retain `UnprovedCoSerialization`.

□ Step 7: Render, validate, and commit the documentation sources

For each substantial Markdown file created by this task, generate a same-basename standalone LaTeX source with Pandoc, compile a same-basename PDF with XeLaTeX, render the PDF pages through Poppler, and inspect them for clipping, missing text, bad glyphs, and blank pages. Regenerate from Markdown after every correction; never edit generated LaTeX or PDF directly. Commit Markdown and LaTeX, but leave PDFs uncommitted as required by project hygiene.

```
$ErrorActionPreference = "Stop"
function Invoke-NativeChecked([string] $tool, [string[]] $arguments) {
    & $tool @arguments
    if ($LASTEXITCODE -ne 0) {
        throw "$tool failed with exit code $LASTEXITCODE"
    }
}

$sourceDocs = @(
    "docs/research/vintage-story-1.22.7-api-evidence.md",
    "docs/architecture/0001-physical-settlement-recovery-threat-model.md",
    "docs/architecture/0002-economically-accrued-liability-index.md",
    "docs/architecture/0003-domain-identifiers-and-financial-time.md",
    "docs/architecture/0004-persistence-sections-and-revisions.md",
    "docs/testing/foundation-engine-smoke.md"
)

$systemTemp = [IO.Path]::GetFullPath([IO.Path]::GetTempPath())
$renderRoot = Join-Path $systemTemp ("fgb-docs-" + [Guid]::NewGuid().ToString("N"))
[IO.Directory]::CreateDirectory($renderRoot) | Out-Null
foreach ($sourceDoc in $sourceDocs) {
    $tex = [IO.Path]::ChangeExtension($sourceDoc, ".tex")
    $pdf = [IO.Path]::ChangeExtension($sourceDoc, ".pdf")
    $stem = [IO.Path]::GetFileNameWithoutExtension($sourceDoc)
    $documentOutput = Join-Path $renderRoot $stem
    $pageOutput = Join-Path $documentOutput "pages"
    [IO.Directory]::CreateDirectory($pageOutput) | Out-Null

    Invoke-NativeChecked "pandoc" @("--standalone", "--from=gfm", "--to=latex", "--output=$tex", $sourceDoc)
    Invoke-NativeChecked "xelatex" @("-interaction=nonstopmode", "-halt-on-error", "-output-directory=$documentOutput", $tex)
    Invoke-NativeChecked "xelatex" @("-interaction=nonstopmode", "-halt-on-error", "-output-directory=$documentOutput", $tex)
    Copy-Item -Force -LiteralPath (Join-Path $documentOutput "$stem.pdf") -Destination $pdf
    Invoke-NativeChecked "pdftoppm" @("-png", "-r", "144", $pdf, (Join-Path $pageOutput "page"))
}
Write-Output "FGB_RENDER_ROOT=$renderRoot"
```

Stop here. Retain the printed render root and inspect every generated `pages/page-*.png` with the image viewer. Record the page count and pass/fail for each document; correct Markdown and repeat the complete render if any page fails. Only after all pages pass, reassign `$renderRoot` to the exact `FGB_RENDER_ROOT` path if the shell changed and run this guarded cleanup and source check:

```
$systemTemp = [IO.Path]::GetFullPath([IO.Path]::GetTempPath())
if ([string]::IsNullOrEmpty($renderRoot)) { throw "The inspected render root is required." }
$resolvedRenderRoot = [IO.Path]::GetFullPath($renderRoot)
$requiredPrefix = $systemTemp.TrimEnd([IO.Path]::DirectorySeparatorChar) + [IO.Path]::DirectorySeparatorChar
if (-not $resolvedRenderRoot.StartsWith($requiredPrefix, [StringComparison]::OrdinalIgnoreCase) -or
    [IO.Path]::GetFileName($resolvedRenderRoot) -notmatch '^fgb-docs-[0-9a-f]{32}$') {
    throw "Refusing to remove an unexpected render directory: $resolvedRenderRoot"
}
[IO.Directory]::Delete($resolvedRenderRoot, $true)
rg -n "Verified public|unproved|Gate result|quarantine|1\..22\..7" docs/research docs/architecture docs/testing
git diff --check
git add docs/research/vintage-story-1.22.7-api-evidence.md `
    docs/research/vintage-story-1.22.7-api-evidence.tex `
    docs/architecture/0001-physical-settlement-recovery-threat-model.md `
    docs/architecture/0001-physical-settlement-recovery-threat-model.tex `
    docs/architecture/0002-economically-accrued-liability-index.md `
    docs/architecture/0002-economically-accrued-liability-index.tex `
    docs/architecture/0003-domain-identifiers-and-financial-time.md `
    docs/architecture/0003-domain-identifiers-and-financial-time.tex `
    docs/architecture/0004-persistence-sections-and-revisions.md `
    docs/architecture/0004-persistence-sections-and-revisions.tex `
    docs/testing/foundation-engine-smoke.md docs/testing/foundation-engine-smoke.tex
git commit -m "docs: record Vintage Story foundation evidence"
```

Expected: the evidence distinguishes verified API facts from runtime observations and unresolved gates.

Task 2: Establish the Project Graph, Test Runner, and Repository Hygiene

Files:

- Create: AGENTS.md
- Create: README.md
- Create: .editorconfig
- Create: .gitattributes
- Create: global.json
- Create: Directory.Build.props
- Create: FirstGearBank.Core/FirstGearBank.Core.csproj
- Create: FirstGearBank.Core/CoreAssemblyMarker.cs
- Create: tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj
- Create: tests/FirstGearBank.Core.Tests/Architecture/CoreAssemblyBoundaryTests.cs
- Create: tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj
- Create: tests/CakeBuild.Tests/CakeBuild.Tests.csproj
- Create: FirstGearBank.Core/packages.lock.json through dotnet restore
- Create: First Gear Bank/packages.lock.json through dotnet restore
- Create: CakeBuild/packages.lock.json through dotnet restore
- Create: tests/FirstGearBank.Core.Tests/packages.lock.json through dotnet restore
- Create: tests/FirstGearBank.Adapter.Tests/packages.lock.json through dotnet restore
- Create: tests/CakeBuild.Tests/packages.lock.json through dotnet restore
- Modify: .gitignore:1
- Modify: First Gear Bank.sln:1
- Modify: First Gear Bank/First Gear Bank.csproj:1

Interfaces:

- Consumes: SDK 10.0.400 and VINTAGE_STORY/VintagestoryAPI.dll version 1.22.7.0.
- Produces: FirstGearBank.Core, FirstGearBank, and three test projects under one reproducible solution.

☐ Step 1: Add the SDK and compiler contracts

Create global.json with:

```
{
  "sdk": {
    "version": "10.0.400",
    "rollForward": "latestPatch"
  },
  "test": {
    "runner": "Microsoft.Testing.Platform"
  }
}
```

Create Directory.Build.props with net10.0, nullable annotations, implicit usings, deterministic builds, LangVersion=14.0, generated-file exclusions, and RestorePackagesWithLockFile=true. Do not globally suppress NuGet audit, nullable, or analyzer warnings.

☐ Step 2: Create the core and test project files before the marker type exists

Use this test-project shape for each xUnit project, changing project references per target:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <Using Include="Xunit" />
    <PackageReference Include="xunit.v3.mtp-v2" Version="4.0.0" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\..\FirstGearBank.Core\FirstGearBank.Core.csproj" />
  </ItemGroup>
</Project>
```

`FirstGearBank.Adapter.Tests` references both core and `First Gear Bank.csproj`; its test-only Vintage Story API reference may copy locally so the runner can load adapter types. The packaged adapter keeps `Private="false"`.

□ Step 3: Write the failing dependency-boundary test

```
/*
 * Protects the engine-independent assembly boundary required by the locked architecture.
 */

using FirstGearBank.Core;

namespace FirstGearBank.Core.Tests.Architecture;

public sealed class CoreAssemblyBoundaryTests
{

    /// Verifies that the banking core cannot acquire an engine reference unnoticed.
    ///
    [Fact]
    public void Core_assembly_references_no_vintage_story_assembly()
    {
        var references = typeof(CoreAssemblyMarker).Assembly.GetReferencedAssemblies();

        Assert.DoesNotContain(
            references,
            reference => reference.Name?.StartsWith("Vintagestory", StringComparison.OrdinalIgnoreCase) == true);
    }

}
```

□ Step 4: Run the test and observe the intended compile failure

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because `CoreAssemblyMarker` does not exist.

□ Step 5: Add the minimal core marker and project reference

```
/*
 * Provides a stable assembly anchor for dependency and packaging verification.
 */

namespace FirstGearBank.Core;

public static class CoreAssemblyMarker
{
}
```

Set the adapter project to `AssemblyName=FirstGearBank`, `RootNamespace=FirstGearBank`, and add a project reference to core. Keep only `VintagestoryAPI.dll` as a non-copying engine reference. Remove the custom publish output path and unused folder item; packaging owns final layout. Expose core internals only to `FirstGearBank.Core.Tests` and the named `FirstGearBank` adapter assembly through `SDK InternalsVisibleTo` items: tests need deliberately corrupt fixtures, while the trusted server adapter needs to construct permission grants after engine authorization. No client assembly is a friend, and invariant-breaking constructors remain nonpublic.

□ Step 6: Add repository instructions and reproducible setup documentation

`AGENTS.md` must direct contributors to the four checked-in standards, locked v03 spec, this plan/roadmap, and the indexed 1.22.7 reference instructions. `README.md` must document .NET 10.0.400, `VINTAGE_STORY`, the exact solution commands, explicit packaging/deployment, the pure-core boundary, and the fact that Phase 1 has no playable banking UI.

Configure UTF-8, final newlines, four-space C# indentation, LF for text, and binary treatment for DLL/PNG/DOCX/PDF/ZIP. Set `dotnet_style_allow_multiple_blank_lines_experimental = true` for C# so Roslyn does not collapse the profile's mandatory three-blank-line callable spacing; the source-standard gate still checks the exact project rule. Extend `.gitignore` with `Artifacts/`, `Releases/`, `TestResults/`, `logs/`, `.vs/`, and `*.zip` without removing existing entries. Ignore generated PDFs only in `docs/research/`, `docs/architecture/`, `docs/testing/`, and `docs/superpowers/plans/`; keep their canonical Markdown and generated LaTeX tracked.

□ **Step 7: Add all projects to the solution and prove the boundary test passes**

```
dotnet sln "First Gear Bank.sln" add "FirstGearBank.Core/FirstGearBank.Core.csproj"
dotnet sln "First Gear Bank.sln" add "tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj"
dotnet sln "First Gear Bank.sln" add "tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj"
dotnet sln "First Gear Bank.sln" add "tests/CakeBuild.Tests/CakeBuild.Tests.csproj"
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: PASS, and FirstGearBank.Core.csproj contains no assembly or package reference to Vintage Story.

□ **Step 8: Build, inspect, and commit the project foundation**

```
dotnet restore "First Gear Bank.sln"
dotnet build "First Gear Bank.sln" -c Release --no-restore
dotnet list FirstGearBank.Core/FirstGearBank.Core.csproj reference
git diff --check
git add AGENTS.md README.md .editorconfig .gitattributes .gitignore global.json Directory.Build.props `
  "First Gear Bank.sln" "First Gear Bank/First Gear Bank.csproj" `
  "First Gear Bank/packages.lock.json" CakeBuild/packages.lock.json FirstGearBank.Core tests
git commit -m "build: establish core and test project foundation"
```

Expected: zero build errors and no Vintagestory.* core reference.

Task 3: Build Deterministic Package and Deployment-Safety Primitives

Files:

- Create: CakeBuild/BuildPaths.cs
- Create: CakeBuild/RepositoryLocator.cs
- Create: CakeBuild/ModManifest.cs
- Create: CakeBuild/Packaging/SemanticVersion.cs
- Create: CakeBuild/Packaging/PackageArchiveLimits.cs
- Create: CakeBuild/Packaging/PackageArchiveWriter.cs
- Create: CakeBuild/Packaging/PackageArchiveInspector.cs
- Create: CakeBuild/Packaging/PackageInspection.cs
- Create: CakeBuild/Packaging/DeploymentPackageSelector.cs
- Create: CakeBuild/Packaging/DeploymentPathValidator.cs
- Create: CakeBuild/Packaging/DeploymentPathValidation.cs
- Create: CakeBuild/Packaging/ValidatedDeploymentDirectory.cs
- Create: CakeBuild/Packaging/IDeploymentFileOperations.cs
- Create: CakeBuild/Packaging/PhysicalDeploymentFileOperations.cs
- Create: CakeBuild/Packaging/PackageDeploymentInstaller.cs
- Create: CakeBuild/Packaging/PackageDeploymentResult.cs
- Create: tests/CakeBuild.Tests/BuildPathsTests.cs
- Create: tests/CakeBuild.Tests/RepositoryLocatorTests.cs
- Create: tests/CakeBuild.Tests/SemanticVersionTests.cs
- Create: tests/CakeBuild.Tests/ModManifestTests.cs
- Create: tests/CakeBuild.Tests/PackageArchiveWriterTests.cs
- Create: tests/CakeBuild.Tests/PackageArchiveInspectorTests.cs
- Create: tests/CakeBuild.Tests/DeploymentPackageSelectorTests.cs
- Create: tests/CakeBuild.Tests/DeploymentPathValidatorTests.cs
- Create: tests/CakeBuild.Tests/PackageDeploymentInstallerTests.cs
- Create: tests/CakeBuild.Tests/TestSupport/TestArchive.cs
- Create: tests/CakeBuild.Tests/TestSupport/TestDeploymentDirectory.cs
- Create: tests/CakeBuild.Tests/TestSupport/TestManifest.cs
- Create: tests/CakeBuild.Tests/TestSupport/FaultInjectingDeploymentFileOperations.cs
- Modify: tests/CakeBuild.Tests/CakeBuild.Tests.csproj:1

Interfaces:

- Consumes: the repository project graph from Task 2 and build-only BCL APIs.
- Produces: tested, Cake-independent path, version, manifest, archive, and deployment-selection logic used by Task 4.

□ **Step 1: Point the build tests at CakeBuild and write the failing path/version tests**

Add a project reference to `../.. /CakeBuild/CakeBuild.csproj`, then add these cases:

```
[Fact]
public void Repository_paths_preserve_the_adapter_directory_with_spaces()
{
    var paths = BuildPaths.FromRepositoryRoot(new DirectoryInfo(@"C:\repo\First Gear Bank"));

    Assert.Equal(
        @"C:\repo\First Gear Bank\First Gear Bank\First Gear Bank.csproj",
        paths.AdapterProject.FullName);
}

[Theory]
[InlineData("1.2.3", VersionComponent.Major, "2.0.0")]
[InlineData("1.2.3", VersionComponent.Minor, "1.3.0")]
[InlineData("1.2.3", VersionComponent.Patch, "1.2.4")]
public void Bump_changes_exactly_one_semantic_component(
    string source,
    VersionComponent component,
    string expected)
{
    Assert.Equal(expected, SemanticVersion.Parse(source).Bump(component).ToString());
}
```

□ Step 2: Run the focused tests and observe missing-type failures

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
```

Expected: FAIL because `BuildPaths`, `SemanticVersion`, and `VersionComponent` do not exist.

□ Step 3: Implement path separation and strict semantic versions

Use these public contracts:

```
public sealed record BuildPaths(
    DirectoryInfo RepositoryRoot,
    FileInfo Solution,
    DirectoryInfo AdapterDirectory,
    FileInfo AdapterProject,
    FileInfo Manifest,
    DirectoryInfo PublishDirectory,
    DirectoryInfo StagingDirectory,
    DirectoryInfo ReleasesDirectory)
{
    public static BuildPaths FromRepositoryRoot(DirectoryInfo repositoryRoot);
}

public enum VersionComponent
{
    Major = 1,
    Minor = 2,
    Patch = 3
}

public readonly record struct SemanticVersion(int Major, int Minor, int Patch)
{
    public static SemanticVersion Parse(string text);
    public SemanticVersion Bump(VersionComponent component);
    public override string ToString();
}
```

`Parse` accepts exactly three nonnegative invariant integer components and rejects whitespace, signs, leading/trailing text, overflow, and extra components. `BuildPaths.FromRepositoryRoot` resolves absolute descendants and verifies that each build-owned path remains under the supplied repository root.

`RepositoryLocator.Locate(DirectoryInfo start)` checks only `start` and its ancestors for `First Gear Bank.sln`; it never recursively scans drives. Tests cover invocation from the repository root, `CakeBuild`, a compiled `bin` descendant, and a directory with no matching ancestor.

□ Step 4: Prove path/version behavior is green

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
```

Expected: PASS for valid bumps and rejection tests for 1, 1.2, 1.2.3.4, -1.2.3, and numeric overflow.

□ Step 5: Write failing manifest and archive-inspection tests

Construct temporary ZIPs in the test process and assert all of these rules:

```
[Fact]
public void Valid_release_requires_both_mod_assemblies_at_the_root()
{
    using var archive = TestArchive.Create(
        ("modinfo.json", TestManifest.ValidBytes),
        ("FirstGearBank.dll", [1, 2, 3]));

    var result = PackageArchiveInspector.Inspect(archive.Path);

    Assert.Contains(result.Errors, error => error.Code == "MissingCoreAssembly");
}

[Theory]
[InlineData("../outside.txt")]
[InlineData("/rooted.txt")]
[InlineData("C:/rooted.txt")]
[InlineData("assets\\firstgearbank\\bad.json")]
public void Unsafe_entry_names_are_rejected(string entryName)
{
    using var archive = TestArchive.Create(
        ("modinfo.json", TestManifest.ValidBytes),
        ("FirstGearBank.dll", [1]),
        ("FirstGearBank.Core.dll", [2]),
        (entryName, [3]));

    Assert.False(PackageArchiveInspector.Inspect(archive.Path).IsValid);
}
```

Add explicit tests for wrong `modid`, wrong game dependency, package/manifest version mismatch, duplicate names under `OrdinalIgnoreCase`, malformed JSON assets, and prohibited `VintagestoryAPI.dll`, Cake DLLs, PDB, `.deps.json`, and `.runtimeconfig.json` entries. Exercise each archive bound at exactly the limit and at limit plus one: 10,000/10,001 entries, 64 KiB/64 KiB plus one manifest bytes, 64 MiB/64 MiB plus one entry bytes, and 256 MiB/256 MiB plus one total declared uncompressed bytes. Also exercise 256 MiB/plus one physical archive bytes, 8 MiB/plus one central-directory bytes, 64 MiB/plus one declared compressed entry bytes, and 256 MiB/plus one total declared compressed bytes. Include a sparse oversized foreign ZIP and prove it is rejected from file length before hashing or central-directory traversal. The fixtures may write central-directory lengths or a sparse file length without allocating the declared payload.

Keep `TestArchive`, `TestDeploymentDirectory`, and `TestManifest` under the test project. They create uniquely named temporary directories, expose exact fixture bytes, and delete only their own resolved directory during `Dispose`.

□ Step 6: Run the archive tests and observe missing behavior

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
```

Expected: FAIL because the manifest codec, archive writer, and inspector are absent.

□ Step 7: Implement strict manifest parsing and deterministic ZIP construction

Use these contracts:

```
public sealed record ModManifest(
    string Type,
    string ModId,
    string Name,
    ImmutableArray<string> Authors,
    string Description,
    SemanticVersion Version,
    ImmutableDictionary<string, string> Dependencies);

public sealed record PackageInspection(
    bool IsValid,
    ModManifest? Manifest,
    ImmutableArray<PackageIssue> Errors,
    string? Sha256Hex);

public static class PackageArchiveLimits
{
    public const long MaximumArchiveBytes = 256L * 1024 * 1024;
    public const int MaximumCentralDirectoryBytes = 8 * 1024 * 1024;
}
```

```

    public const long MaximumCompressedEntryBytes = 64L * 1024 * 1024;
    public const long MaximumTotalCompressedBytes = 256L * 1024 * 1024;
}

public static class PackageArchiveWriter
{
    public static void Write(
        DirectoryInfo stagingDirectory,
        FileInfo destination,
        SemanticVersion expectedVersion);
}

public static class PackageArchiveInspector
{
    public static PackageInspection Inspect(
        FileInfo archive,
        SemanticVersion? expectedVersion = null);
}

```

Parse with `System.Text.Json`, require `type=code`, `modid=firstgearbank`, and `dependencies.game=1.22.7`, and reject missing/extra root placement for required files. Enumerate staging files with ordinal normalized relative paths, reject unsafe names and filesystem reparse points before opening the destination, and assign every ZIP entry the fixed timestamp 2000-01-01T00:00:00Z. Bound inspection before decompression to 10,000 entries, a 64 KiB root manifest, 64 MiB per entry, 256 MiB total uncompressed content, and the physical/compressed/central-directory limits above. Read file length before opening `ZipArchive`; validate central-directory range and every declared compressed range against that length before entry content is opened. Hash a file only after all structural bounds pass, and do not hash unrelated foreign archives during deployment selection. Write to a uniquely named same-directory temporary ZIP, inspect and hash it, and only then atomically replace the destination. Before replacing an existing release path, inspect it and refuse replacement unless its root manifest also identifies `firstgearbank`; preserve both files if replacement cannot be completed safely.

□ Step 8: Verify deterministic archives and all rejection cases

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
```

Expected: PASS, including byte-identical SHA-256 hashes for two archives built from identical staged bytes.

□ Step 9: Write failing deployment-selection, path, and transaction tests

```

[Fact]
public void Selection_uses_the_internal_mod_id_and_preserves_unreadable_archives()
{
    using var directory = TestDeploymentDirectory.Create(
        ("misleading-firstgearbank.zip", TestArchive.Foreign),
        ("unrelated-name.zip", TestArchive.FirstGearBank),
        ("damaged.zip", TestArchive.Unreadable));

    var result = DeploymentPackageSelector.FindExistingMatches(directory.Path, "firstgearbank");

    Assert.Equal(["unrelated-name.zip"], result.MatchingArchives.Select(file => file.Name));
    Assert.Contains(result.Warnings, warning => warning.Path.Name == "damaged.zip");
}

```

Add table-driven `DeploymentPathValidatorTests` for a relative path, filesystem root, repository root/descendant, publish/stage/release directory, a normal external Mods directory, and every ancestor reparse point whose resolved target enters a forbidden tree. Every invalid result has no validated-directory token. Use the file-operations seam rather than requiring OS symlink privileges.

`PackageDeploymentInstallerTests` use uniquely scoped temporary directories and a fault-injecting file-operations adapter. For each copy, source-hash, temporary-package inspection, existing-package backup move, final rename, destination-hash, final inspection, and backup-cleanup boundary, inject one failure and assert that no unconfirmed archive is deleted, every old confirmed package is restored when installation did not complete, the source is unchanged, and any retained recovery file is reported. Also test hash mismatch, an existing exact-match package, misleading filenames, foreign/unreadable ZIPs, and successful removal of confirmed backups only after the new ZIP verifies. Prove the install API has no raw-`DirectoryInfo` overload, that a reparse target changed after initial validation fails revalidation, and that every invalid/stale token path causes zero mutating file-operation calls.

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
```

Expected: FAIL because deployment selection by bounded internal manifest identity is not implemented.

□ Step 10: Implement and verify safe selection

```
public sealed record DeploymentSelection(
    ImmutableArray<FileInfo> MatchingArchives,
    ImmutableArray<DeploymentWarning> Warnings);

public static class DeploymentPackageSelector
{
    public static DeploymentSelection FindExistingMatches(
        DirectoryInfo modsDirectory,
        string expectedModId);
}

public static class DeploymentPathValidator
{
    public static DeploymentPathValidation Validate(
        BuildPaths buildPaths,
        DirectoryInfo candidate,
        IDeploymentFileOperations fileOperations);
}

public sealed record DeploymentPathValidation(
    bool IsValid,
    ValidatedDeploymentDirectory? Directory,
    ImmutableArray<DeploymentIssue> Issues);

public sealed class ValidatedDeploymentDirectory
{
    public DirectoryInfo Path { get; }
    public string ResolvedPath { get; }

    internal ValidatedDeploymentDirectory(
        DirectoryInfo path,
        string resolvedPath,
        ImmutableArray<string> forbiddenRoots,
        ImmutableArray<string> resolvedAncestors);
    internal ImmutableArray<DeploymentIssue> Revalidate(IDeploymentFileOperations fileOperations);
}

public static class PackageDeploymentInstaller
{
    public static PackageDeploymentResult Install(
        FileInfo sourceArchive,
        ValidatedDeploymentDirectory modsDirectory,
        string expectedModId,
        IDeploymentFileOperations fileOperations);
}
```

Production supplies `PhysicalDeploymentFileOperations`; tests supply the fault-injecting implementation. Both lexical absolute paths and every existing ancestor's resolved reparse target must remain outside the repository and all build-owned trees. Only the validator's internal constructor can create the token, and the token captures the normalized destination, forbidden roots, and resolved-ancestor fingerprint. The installer revalidates that token immediately before its first mutation and again before final rename; it has no raw-path overload. It uses uniquely named same-directory temporary/backup files, restores in reverse order on failure, and returns every retained recovery path in `PackageDeploymentResult`. It never accepts a broad directory based only on string prefix; path comparison is separator-aware and case-insensitive on Windows.

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
```

Expected: PASS; malformed, unreadable, missing-manifest, and foreign archives are never returned for deletion.

□ Step 11: Run all build-logic tests and commit

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
git diff --check
git add CakeBuild/BuildPaths.cs CakeBuild/RepositoryLocator.cs CakeBuild/ModManifest.cs CakeBuild/Packaging `
    tests/CakeBuild.Tests
git commit -m "build: add deterministic package safety primitives"
```

Task 4: Replace the Unsafe Cake Task Graph and Correct Mod Metadata

Files:

- Create: `Directory.Packages.props`
- Create: `CakeBuild/Tasks/ValidateEnvironmentTask.cs`
- Create: `CakeBuild/Tasks/ValidateMetadataTask.cs`
- Create: `CakeBuild/Tasks/RestoreTask.cs`
- Create: `CakeBuild/Tasks/TestTask.cs`
- Create: `CakeBuild/Tasks/StageTask.cs`
- Create: `CakeBuild/Tasks/PackageTask.cs`
- Create: `CakeBuild/Tasks/VerifyPackageTask.cs`
- Create: `CakeBuild/Tasks/DefaultTask.cs`
- Modify: `CakeBuild/CakeBuild.csproj:1`
- Modify: `CakeBuild/BuildContext.cs:1`
- Modify: `CakeBuild/Program.cs:1`
- Modify: `CakeBuild/Tasks/BuildTask.cs:1`
- Modify: `CakeBuild/Tasks/ValidateJsonTask.cs:1`
- Modify: `CakeBuild/Tasks/DeployTask.cs:1`
- Delete: `CakeBuild/Tasks/PackageModTask.cs`
- Delete: `CakeBuild/Tasks/ZipModFolder.cs`
- Modify: `First Gear Bank/modinfo.json:1`
- Modify: `tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj:1`
- Modify: `tests/FirstGearBank.Core.Tests/packages.lock.json:1`
- Modify: `tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj:1`
- Modify: `tests/FirstGearBank.Adapter.Tests/packages.lock.json:1`
- Modify: `tests/CakeBuild.Tests/CakeBuild.Tests.csproj:1`
- Modify: `tests/CakeBuild.Tests/packages.lock.json:1`
- Modify: `FirstGearBank.Core/packages.lock.json:1`
- Modify: `First Gear Bank/packages.lock.json:1`
- Modify: `CakeBuild/packages.lock.json:1`
- Modify: `build.ps1:1`
- Modify: `build.py:1`
- Modify: `README.md:1`

Interfaces:

- Consumes: the tested build primitives from Task 3.
- Produces: unique `Default`, `Package`, `VerifyPackage`, and explicit `Deploy` targets with no implicit destructive action.

☐ Step 1: Capture the current red integration baseline

```
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=Build
```

Expected: FAIL while looking for `First_Gear_Bank/modinfo.json`, proving that one identifier was incorrectly used as a directory, project name, and mod ID.

☐ Step 2: Centralize final package versions and remove build-only game dependencies

Create:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Cake.Frosting" Version="6.2.0" />
    <PackageVersion Include="xunit.v3.mtp-v2" Version="4.0.0" />
  </ItemGroup>
</Project>
```

Remove explicit package versions from all project files. `CakeBuild.csproj` keeps only `Cake.Frosting` and removes `Cake.Json`, `Newtonsoft.Json`, and `VintagestoryAPI`; Task 3's `System.Text.Json` manifest model replaces them.

Regenerate the committed lock files after this intentional dependency change:

```
dotnet restore "First Gear Bank.sln" --force-evaluate
```

☐ Step 3: Correct the source manifest without inventing attribution

```
{
  "type": "code",
  "modid": "firstgearbank",
  "name": "First Gear Bank of the Old Roads",
  "authors": [
    "Unknown"
  ],
  "description": "A server-authoritative bank and economy for Vintage Story.",
  "version": "1.0.0",
  "dependencies": {
    "game": "1.22.7"
  }
}
```

Keep `Unknown` because neither the repository nor the approved specification names a release author.

□ Step 4: Implement the unique nondestructive task graph

Use this exact dependency graph:

```
ValidateEnvironment -> ValidateMetadata -> ValidateJson -> Restore -> Build -> Test -> Default
                                                    ^-> Stage -> Package -> VerifyPackage -> Deploy
```

Default depends only on Test. Stage, Package, VerifyPackage, and Deploy run only when explicitly targeted. BuildContext owns resolved BuildPaths, configuration, optional bump component, and optional deploy directory; it does not read a manifest in its constructor. It resolves the repository through RepositoryLocator from the current working directory, with an optional validated --repositoryRoot override for IDE invocation.

Use these target names exactly:

```
[TaskName("Default")]
[IsDependentOn(typeof(TestTask))]
public sealed class DefaultTask : FrostingTask<BuildContext>
{
}

[TaskName("VerifyPackage")]
[IsDependentOn(typeof(PackageTask))]
public sealed class VerifyPackageTask : FrostingTask<BuildContext>
{
    public override void Run(BuildContext context)
    {
        var inspection = PackageArchiveInspector.Inspect(
            context.ReleaseArchive,
            context.PackageVersion);

        if (!inspection.IsValid)
        {
            throw new CakeException(string.Join(Environment.NewLine, inspection.Errors));
        }
    }
}
```

□ Step 5: Stage only the approved package root

StageTask cleans only Artifacts/stage/firstgearbank, publishes the adapter to Artifacts/publish, copies only FirstGearBank.dll, FirstGearBank.Core.dll, modinfo.json, assets/**, and optional modicon.png, and writes the bumped version only into the staged manifest. It rejects a publish output containing either required DLL zero or multiple times.

PackageTask creates Releases/firstgearbank_<version>.zip through PackageArchiveWriter; it never cleans the complete Releases directory.

□ Step 6: Implement explicit hash-verified ZIP deployment

Resolve the default only inside DeployTask:

```
var appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
var defaultModsDirectory = Path.Combine(
    appData,
    "StoryForge",
    "installations",
    "working_test_world",
    "Mods");
```

Allow `--deployDirectory=<absolute path>`. Reject a relative path, filesystem root, repository descendant, or any build-owned source/stage/release directory. `DeployTask` must obtain a successful `DeploymentPathValidation.Directory` token and pass that token—not the raw option string or `DirectoryInfo`—to `PackageDeploymentInstaller`; any invalid result returns before a mutating file operation. Validate the source ZIP, copy it to a uniquely named temporary file inside the resolved Mods directory, compare SHA-256, and inspect only bounded root `modinfo.json` entries when identifying existing ZIPs. Move confirmed `firstgearbank` matches to unique same-directory backup names, atomically rename the temporary file, recheck SHA-256 and package validity, then remove only those confirmed backups. If installation or verification fails, restore the backups; if cleanup fails after success, retain the backup and warn. Open File Explorer only after verified success. Warn and retain every unreadable or foreign ZIP. Never recursively delete the Mods directory or any unpacked directory.

□ Step 7: Simplify wrapper defaults and remove repository-wide staging

Update usage text so no-argument invocation means `Default`, package verification uses `--target=VerifyPackage`, and deployment uses `--target=Deploy`. Remove `--commit`, Codex invocation, temporary prompt files, and every `git add -A` path from both wrappers. If a caller supplies the removed option, fail with a concise migration message telling them to inspect and commit an explicit path set themselves. The wrappers build; they never decide which repository changes belong in a commit.

□ Step 8: Run focused build tests and all task smoke tests

```
dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=Default
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=VerifyPackage
if (rg -n "git add -A|codex" build.py build.ps1) {
    throw "Build wrappers still contain repository-wide commit behavior."
}
$pythonCommitOutput = & python -B build.py --commit=test 2>&1
if ($LASTEXITCODE -eq 0 -or $pythonCommitOutput -notmatch "removed") {
    throw "build.py did not reject the removed --commit option."
}
$powershellCommitOutput = & pwsh -NoProfile -File .\build.ps1 --commit=test 2>&1
if ($LASTEXITCODE -eq 0 -or $powershellCommitOutput -notmatch "removed") {
    throw "build.ps1 did not reject the removed --commit option."
}
}
```

Expected: both targets succeed; `Default` creates no release and performs no deployment; `VerifyPackage` produces one validated ZIP with both mod DLLs and no game/build DLL.

□ Step 9: Verify repeated packaging and commit

```
$first = (Get-FileHash -Algorithm SHA256 Releases/firstgearbank_1.0.0.zip).Hash
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=VerifyPackage
$second = (Get-FileHash -Algorithm SHA256 Releases/firstgearbank_1.0.0.zip).Hash
if ($first -ne $second) { throw "Package is not reproducible." }
git diff --check
git add Directory.Packages.props CakeBuild "First Gear Bank/modinfo.json" build.ps1 build.py README.md `
    "First Gear Bank/packages.lock.json" FirstGearBank.Core/packages.lock.json `
    tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj `
    tests/FirstGearBank.Core.Tests/packages.lock.json `
    tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj `
    tests/FirstGearBank.Adapter.Tests/packages.lock.json tests/CakeBuild.Tests
git commit -m "build: make packaging reproducible and deployment explicit"
```

Expected: hashes are identical and `git status --short` contains no release artifact because build output is ignored.

Task 5: Add Domain Identifiers, Fixed-Point Money, and Financial Instants

Files:

- Create: `FirstGearBank.Core/Domain/BankCurrency.cs`
- Create: `FirstGearBank.Core/Domain/BankErrorCode.cs`
- Create: `FirstGearBank.Core/Domain/BankFailure.cs`
- Create: `FirstGearBank.Core/Domain/BankResult.cs`
- Create: `FirstGearBank.Core/Domain/Identifiers.cs`
- Create: `FirstGearBank.Core/Domain/BankMutationDomains.cs`
- Create: `FirstGearBank.Core/Domain/BankRevisionVector.cs`
- Create: `FirstGearBank.Core/Domain/BankInvocationKind.cs`

- Create: FirstGearBank.Core/Domain/BankInvocationKey.cs
- Create: FirstGearBank.Core/Domain/BankInvocationIntent.cs
- Create: FirstGearBank.Core/Domain/CommandPayloadDigest.cs
- Create: FirstGearBank.Core/Domain/EvidenceDigest.cs
- Create: FirstGearBank.Core/Money/BankUnits.cs
- Create: FirstGearBank.Core/Money/MoneyLimits.cs
- Create: FirstGearBank.Core/Money/MoneyMath.cs
- Create: FirstGearBank.Core/Money/MoneyFormatter.cs
- Create: FirstGearBank.Core/Time/FinancialInstant.cs
- Create: FirstGearBank.Core/Time/FinancialTimestamp.cs
- Create: FirstGearBank.Core/Diagnostics/BankLogLevel.cs
- Create: FirstGearBank.Core/Diagnostics/IBankLog.cs
- Create: FirstGearBank.Core/Security/IEntropySource.cs
- Create: FirstGearBank.Core/Security/CryptographicEntropySource.cs
- Create: tests/FirstGearBank.Core.Tests/Domain/IdentifierTests.cs
- Create: tests/FirstGearBank.Core.Tests/Domain/BankRevisionVectorTests.cs
- Create: tests/FirstGearBank.Core.Tests/Domain/BankInvocationKeyTests.cs
- Create: tests/FirstGearBank.Core.Tests/Domain/CommandPayloadDigestTests.cs
- Create: tests/FirstGearBank.Core.Tests/Money/BankUnitsTests.cs
- Create: tests/FirstGearBank.Core.Tests/Money/MoneyMathTests.cs
- Create: tests/FirstGearBank.Core.Tests/Money/MoneyFormatterTests.cs
- Create: tests/FirstGearBank.Core.Tests/Time/FinancialInstantTests.cs

Interfaces:

- Consumes: no engine type and no culture-sensitive global state.
- Produces: the serialized value types used by journal, coordinator, settlement, persistence, and every later product.

□ Step 1: Write failing money conversion and overflow tests

```
[Theory]
[InlineData(0.5, 0)]
[InlineData(1.5, 2)]
[InlineData(-0.5, 0)]
[InlineData(-1.5, -2)]
public void Rate_result_conversion_rounds_midpoints_to_even(double exactBankUnits, long expectedUnits)
{
    var result = MoneyMath.RoundRateResultToUnits(exactBankUnits);

    Assert.True(result.IsSuccess);
    Assert.Equal(expectedUnits, result.Value.Value);
}

[Fact]
public void Checked_add_rejects_long_overflow()
{
    var result = new BankUnits(long.MaxValue).Add(new BankUnits(1));

    Assert.Equal(BankErrorCode.ArithmeticOverflow, result.Failure?.Code);
}
```

Add cases for exact one and quarter gear, negative values, NaN, both infinities, long.MinValue negation, exact-input overprecision, and the customer cap 2_147_483_647_000_000.

□ Step 2: Run the money tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the money and result types do not exist.

□ Step 3: Implement typed results and checked fixed-point arithmetic

Use these contracts:

```
public enum BankCurrency : byte
{
    Rusty = 1,
    Temporal = 2
}
```

```

public sealed record BankFailure(BankErrorCode Code, string Diagnostic);

public sealed class BankResult<T>
{
    public bool IsSuccess { get; }
    public T Value { get; }
    public BankFailure? Failure { get; }

    public static BankResult<T> Success(T value);
    public static BankResult<T> Failed(BankErrorCode code, string diagnostic);
}

public readonly record struct BankUnits(long Value)
{
    public const long UnitsPerGear = 1_000_000;

    public BankResult<BankUnits> Add(BankUnits other);
    public BankResult<BankUnits> Subtract(BankUnits other);
    public BankResult<BankUnits> Negate();
}

public static class MoneyLimits
{
    public const long MaximumCustomerBalanceGears = int.MaxValue;
    public const long MaximumCustomerBalanceUnits = 2_147_483_647_000_000;
}

public static class MoneyMath
{
    public static BankResult<BankUnits> ParseExactGears(ReadOnlySpan<char> text);
    public static BankResult<BankUnits> RoundRateResultToUnits(double exactBankUnits);
    public static BankResult<BankUnits> ValidateCustomerBalance(BankUnits units);
}

```

Use invariant decimal parsing for player/configured exact amounts and reject more than six meaningful fractional digits. Use checked arithmetic and return typed failures rather than wrapping. Floating-rate calculations pass their exact bank-unit result to the central converter, which rejects nonfinite/out-of-range input and applies `MidpointRounding.ToEven` once.

Freeze the foundation values explicitly; later phases may append but never renumber them:

```

public enum BankErrorCode : ushort
{
    None = 0,
    InvalidAmount = 1,
    AmountOverprecision = 2,
    NonFiniteValue = 3,
    ArithmeticOverflow = 4,
    InsufficientFunds = 5,
    CustomerBalanceCapExceeded = 6,
    NegativeCustomerBalance = 7,
    InvalidFinancialInstant = 8,
    AccountNotFound = 9,
    InvalidConfiguration = 10,
    MalformedLedgerAccount = 11,
    UnbalancedJournal = 12,
    InvalidJournalSequence = 13,
    DuplicateOperation = 14,
    JournalIntegrityFailure = 15,
    InactiveRequestScope = 16,
    UnexpectedRequestSequence = 17,
    RequestPayloadMismatch = 18,
    AlreadyProcessedResponseExpired = 19,
    DomainUnavailable = 20,
    CorruptState = 21,
    UnsupportedSchema = 22,
    InvalidSettlementEvidence = 23,
    PersistenceFailure = 24,
    UndeclaredMutationDomain = 25,
    IdentifierContractViolation = 26
}

```

Player-facing text is not stored in this enum; later localization maps codes to prose.

□ Step 4: Run the money tests and verify they pass

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: PASS with no culture-dependent result under en-US, de-DE, and invariant test cultures.

□ Step 5: Write failing formatting and financial-instant tests

```
[Theory]
[InlineData(0, "1")]
[InlineData(3, "1.235")]
[InlineData(6, "1.234568")]
public void Rusty_display_rounds_without_changing_ledger_units(int precision, string expected)
{
    var units = new BankUnits(1_234_567);

    Assert.Equal(expected, MoneyFormatter.FormatRusty(units, precision));
    Assert.Equal(1_234_567, units.Value);
}

[Fact]
public void Whole_month_tenor_addition_is_exact()
{
    var start = FinancialInstant.Create(41, 0.375m).Value;

    Assert.Equal(
        FinancialInstant.Create(53, 0.375m).Value,
        start.AddWholeMonths(12).Value);
}
```

Add precision bounds 0..6, negative display, temporal trailing-zero trimming, financial fraction bounds, ordering, overflow, invariant G29 fraction serialization, and conversion to financial years only at the math boundary.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because formatting and normalized financial-time behavior are not implemented.

□ Step 6: Implement presentation-only formatting and normalized time

```
public readonly record struct FinancialInstant : IComparable<FinancialInstant>
{
    public long CompletedMonths { get; }
    public decimal FractionOfMonth { get; }

    public static BankResult<FinancialInstant> Create(
        long completedMonths,
        decimal fractionOfMonth);

    public BankResult<FinancialInstant> AddWholeMonths(int months);
    public double ToFinancialYears();
    public int CompareTo(FinancialInstant other);
}

public readonly record struct FinancialTimestamp
{
    public FinancialInstant Financial { get; }
    public double WorldCalendarTotalDays { get; }

    public static BankResult<FinancialTimestamp> Create(
        FinancialInstant financial,
        double worldCalendarTotalDays);
}
```

Create accepts only completedMonths >= 0 and 0 <= fraction < 1. FinancialTimestamp construction rejects a nonfinite world-calendar value through a factory. The formatter uses decimal division and ties-to-even presentation rounding but never returns a modified BankUnits value.

□ Step 7: Write failing identifier and revision tests

Test all identifier types named below for 16-byte binary roundtrip and lowercase 32-hex diagnostic format. Test that a PlayerId preserves exact case/content, rejects empty and overlong UTF-8 input, and has no public display conversion. Test CommandPayloadDigest for exact 32-byte construction, defensive copying, structural equality, fixed-time comparison, and command-kind/schema/payload domain separation. Freeze NoticeId.ForAdminCorrection with operation bytes

000102030405060708090a0b0c0d0e0f: SHA-256 over the ASCII domain FGB/ADMIN-CORRECTION-NOTICE/1\0 followed by those canonical operation bytes yields the first-16-byte notice ID 8c887b94a1b027db46e160bf54205382. A different operation or derivation domain must differ. Test EvidenceDigest as an exact structural 32-byte SHA-256 wrapper with defensive copies and fixed-time comparison.

```
[Fact]
public void Only_changed_subsystem_revisions_advance()
{
    var before = BankRevisionVector.Initial;

    var after = before.Advance(BankMutationDomains.Finance | BankMutationDomains.Requests);

    Assert.Equal(1, after.Global.Value);
    Assert.Equal(1, after.Finance.Value);
    Assert.Equal(1, after.Requests.Value);
    Assert.Equal(0, after.Registry.Value);
    Assert.Equal(0, after.Branches.Value);
    Assert.Equal(0, after.Recovery.Value);
}
```

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the canonical identifier and revision contracts do not exist.

□ Step 8: Implement canonical identifiers and versioned stable hashes

Create distinct record structs for:

```
WorldInstanceId, PlayerId, OperationId, CommandId, RequestScopeNonce, RequestKey,
CdId, SettlementId, NoticeId, BranchId, RegistryEpochId, AutomaticOperationKey
```

GUID-backed IDs serialize as 16 canonical RFC-4122-order bytes in persistence and lowercase `n` text in diagnostics. `PlayerId` stores the exact authenticated engine UID with a 256-byte UTF-8 bound. `RequestKey` contains player, scope, and a strictly positive signed 64-bit sequence. `AutomaticOperationKey` contains a bounded ASCII domain/version and stable canonical identifier; it never calls process-randomized `string.GetHashCode`. `NoticeId.ForAdminCorrection` performs the exact domain-separated SHA-256 derivation tested above and copies the first 16 digest bytes without GUID version/variant bit rewriting. `EvidenceDigest` retains all 32 SHA-256 bytes and never uses immutable-array reference equality.

Every journal-producing coordinator invocation carries exactly one typed key:

```
public enum BankInvocationKind : byte
{
    AuthenticatedRequest = 1,
    AutomaticOperation = 2
}

public readonly record struct BankInvocationKey
{
    public BankInvocationKind Kind { get; }
    public RequestKey? Request { get; }
    public AutomaticOperationKey? Automatic { get; }

    public static BankInvocationKey ForRequest(RequestKey request);
    public static BankInvocationKey ForAutomatic(AutomaticOperationKey automatic);
}

public enum BankInvocationIntentKind : byte
{
    Caller = 1,
    AutomaticOperation = 2
}

public readonly record struct BankInvocationIntent
{
    public BankInvocationIntentKind Kind { get; }
    public AutomaticOperationKey? AutomaticOperation { get; }
    public CommandPayloadDigest? AutomaticPayloadDigest { get; }

    public static BankInvocationIntent Caller { get; }
    public static BankInvocationIntent ForAutomatic(
        AutomaticOperationKey operation,
        CommandPayloadDigest payloadDigest);
}
```

The factories enforce one-and-only-one representation. Canonical serialization emits the kind byte followed only by that variant's bytes. `CommandPayloadDigest` is the structural 32-byte, domain/schema-separated SHA-256 value used by both request and automatic intents. It hashes `FGB/COMMAND-PAYLOAD/1\0`, bounded command-kind UTF-8 bytes, schema version, and bounded canonical payload bytes using the framing frozen in Task 9. Tests reject empty, dual, or mismatched representations and freeze both variant encodings.

Use one revision vector:

```
[Flags]
public enum BankMutationDomains : ushort
{
    None = 0,
    Finance = 1 << 0,
    Requests = 1 << 1,
    Registry = 1 << 2,
    Delivery = 1 << 3,
    Settlements = 1 << 4,
    Branches = 1 << 5,
    Configuration = 1 << 6,
    Recovery = 1 << 7
}

public sealed record BankRevisionVector(
    BankRevision Global,
    SubsystemRevision Finance,
    SubsystemRevision Requests,
    SubsystemRevision Registry,
    SubsystemRevision Delivery,
    SubsystemRevision Settlements,
    SubsystemRevision Branches,
    SubsystemRevision Configuration,
    SubsystemRevision Recovery)
{
    public static BankRevisionVector Initial { get; }
    public BankRevisionVector Advance(BankMutationDomains changedDomains);
}
```

Define the production entropy boundary at the same time:

```
public interface IEntropySource
{
    void Fill(Span<byte> destination);
}

public sealed class CryptographicEntropySource : IEntropySource
{
    public void Fill(Span<byte> destination);
}
```

`CryptographicEntropySource` fills caller-owned spans through `RandomNumberGenerator.Fill` with no intermediate allocation. Deterministic entropy sources remain test-only; production wiring never accepts a seed.

Define the caller-owned logging boundary at the same time:

```
public enum BankLogLevel : byte
{
    Info = 1,
    Debug = 2,
    Warning = 3,
    Critical = 4
}

public interface IBankLog
{
    void Write(
        BankLogLevel level,
        string component,
        string message,
        Exception? exception = null);
}
```

The core emits through this interface only; it never creates a file or calls an engine logger.

□ Step 9: Verify all primitive tests and commit

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
git diff --check
git add FirstGearBank.Core/Domain FirstGearBank.Core/Money FirstGearBank.Core/Time `
    FirstGearBank.Core/Diagnostics FirstGearBank.Core/Security tests/FirstGearBank.Core.Tests/Domain `
    tests/FirstGearBank.Core.Tests/Money tests/FirstGearBank.Core.Tests/Time
git commit -m "feat: add fixed-point banking primitives"
```

Task 6: Parse and Validate the Complete Static JSONC Configuration

Files:

- Create: FirstGearBank.Core/Configuration/InterestTimeBasis.cs
- Create: FirstGearBank.Core/Configuration/RiskFreeRateModel.cs
- Create: FirstGearBank.Core/Configuration/RecipientSelectionMode.cs
- Create: FirstGearBank.Core/Configuration/BankConfiguration.cs
- Create: FirstGearBank.Core/Configuration/BankConfigurationDefaults.cs
- Create: FirstGearBank.Core/Configuration/ConfigurationDiagnostic.cs
- Create: FirstGearBank.Core/Configuration/ConfigurationParseResult.cs
- Create: FirstGearBank.Core/Configuration/ValidatedEconomicConfiguration.cs
- Create: FirstGearBank.Core/Configuration/BankConfigurationJsonc.cs
- Create: FirstGearBank.Core/Configuration/JsoncSyntaxDocument.cs
- Create: FirstGearBank.Core/Configuration/IEconomicConfigurationGate.cs
- Create: FirstGearBank.Core/Configuration/IBankConfigurationStore.cs
- Create: First Gear Bank/Configuration/firstgearbank.jsonc.template
- Create: tests/FirstGearBank.Core.Tests/Configuration/ConfigurationDefaultsTests.cs
- Create: tests/FirstGearBank.Core.Tests/Configuration/ConfigurationDomainTests.cs
- Create: tests/FirstGearBank.Core.Tests/Configuration/JsoncRoundTripTests.cs
- Create: tests/FirstGearBank.Core.Tests/Configuration/LegacyConfigurationMigrationTests.cs
- Modify: First Gear Bank/First Gear Bank.csproj:1

Interfaces:

- Consumes: exact money parsing and the enum/record identifiers from Task 5.
- Produces: a fully typed configuration candidate, field-specific diagnostics, a lossless JSONC syntax document, and the derived-economic-validation gate consumed by the financial-core plan.

□ Step 1: Write failing locked-default tests

```
[Fact]
public void Defaults_match_the_locked_v03_configuration()
{
    var value = BankConfigurationDefaults.Value;

    Assert.Equal(InterestTimeBasis.InGame, value.InterestTimeBasis);
    Assert.Equal(RiskFreeRateModel.Cir, value.RiskFreeRateModel);
    Assert.Equal(3.0, value.TargetAnnualizedRiskFreeRate);
    Assert.Equal(1.0, value.Cir.MeanReversionSpeed);
    Assert.Equal(0.12, value.Cir.Volatility);
    Assert.Equal(3, value.Currency.RustyGearDisplayPrecision);
    Assert.Equal(new BankUnits(250_000), value.Cd.MinimumRustyGearPrincipal);
    Assert.Equal([1, 3, 6, 12], value.Cd.TenorsMonths);
    Assert.Equal(RecipientSelectionMode.ExactName, value.RecipientSelectionMode);
    Assert.Equal(1.0, value.TransferCooldownSeconds);
    Assert.Equal(2.0, value.Charter.ArrivalMinimumDays);
    Assert.Equal(5.0, value.Charter.ArrivalMaximumDays);
    Assert.Equal(3.0, value.BankerReplacement.MinimumDays);
    Assert.Equal(7.0, value.BankerReplacement.MaximumDays);
    Assert.Equal(32, value.MinimumCharterBranchSpacingBlocks);
    Assert.Equal(0.15, value.NaturalBranches.TraderCompanionProbability);
    Assert.False(value.NaturalBranches.BackfillExistingTraderLocations);
    Assert.Equal(10, value.Statements.RecentTransactionsOnPrintedStatement);
}
```

Also assert base-spread 0.30/0.395, level shock gaussian/0/0.05, slope shock gaussian/0/0.04, and liquidity 0.25/0.15/0.10/1.0/0.10 in the same test file.

□ Step 2: Run the default tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the configuration records and defaults do not exist.

□ Step 3: Implement immutable records for every section 18.1 field

Use these top-level contracts and nested records with exactly the representative field names:

```
public sealed record BankConfiguration(  
    InterestTimeBasis InterestTimeBasis,  
    RiskFreeRateModel RiskFreeRateModel,  
    double TargetAnnualizedRiskFreeRate,  
    CirConfiguration Cir,  
    CurrencyConfiguration Currency,  
    CdConfiguration Cd,  
    RecipientSelectionMode RecipientSelectionMode,  
    double TransferCooldownSeconds,  
    CharterConfiguration Charter,  
    BankerReplacementConfiguration BankerReplacement,  
    int MinimumCharterBranchSpacingBlocks,  
    NaturalBranchesConfiguration NaturalBranches,  
    StatementsConfiguration Statements);  
  
public static class BankConfigurationDefaults  
{  
    public static BankConfiguration Value { get; }  
}
```

Copy every default literally from section 18.1. Store the CD minimum as `BankUnits` after exact decimal conversion and store tenor months as an immutable array.

□ Step 4: Verify defaults, then write failing field-domain tests

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Add one theory row at each accepted/rejected boundary from section 18: enum case sensitivity; target q ; CIR κ and σ ; Gaussian distribution/zero average/nonnegative variation; spread scale/exponent; funding ratio; adjustment bounds; strictly positive width/half-life; exact positive minimum principal; unique positive tenors; probability; precision; cooldown; spacing; ordered positive timers; and printed count 0..100.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL on the first unimplemented field-domain validation.

□ Step 5: Implement field-local parsing and diagnostics

```
public sealed record ConfigurationDiagnostic(  
    string JsonPath,  
    ConfigurationDiagnosticCode Code,  
    string Message);  
  
public sealed record ConfigurationParseResult(  
    BankConfiguration Candidate,  
    JsoncSyntaxDocument Source,  
    ImmutableArray<ConfigurationDiagnostic> Diagnostics,  
    bool UsesLegacyStochasticSpreadKey);  
  
public static class BankConfigurationJsonc  
{  
    public static ConfigurationParseResult Parse(string jsonc);  
}
```

Parse with comments and trailing commas enabled, but read every property explicitly so exact enum spellings remain closed. An invalid field emits one diagnostic containing its exact JSON path and uses only that field's documented default. Unknown properties are retained in the syntax document and reported once as `UnknownProperty`; they are not executed or silently copied into typed state. A malformed document returns the complete documented defaults and one root diagnostic.

□ Step 6: Run all domain-boundary tests

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: PASS; unrelated valid fields survive every single-field invalid fixture.

□ Step 7: Write failing lossless JSONC ownership tests

```
[Fact]
public void Reading_valid_comment_free_json_does_not_request_a_rewrite()
{
    var result = BankConfigurationJsonc.Parse("{\"InterestTimeBasis\":\"InGame\"}");

    Assert.False(result.Source.RequiresOwnedWrite);
}

[Fact]
public void Owned_scalar_edit_preserves_comments_unknown_properties_and_line_endings()
{
    const string source = "{\r\n // operator note\r\n  \"TransferCooldownSeconds\": 1.0,\r\n  \"OperatorTag\": \"keep\"\r\n}\r\n";
    var document = BankConfigurationJsonc.Parse(source).Source;

    var edited = document.ReplaceScalar("$.TransferCooldownSeconds", "2.5");

    Assert.Contains("// operator note\r\n", edited);
    Assert.Contains("\"OperatorTag\": \"keep\"", edited);
    Assert.Contains("\"TransferCooldownSeconds\": 2.5", edited);
}
```

Add tests for LF, CRLF, trailing commas, escaped strings, comment markers inside strings, comments before/after a value, and no rewrite on ordinary successful load. Also write the full legacy `StochasticCDSspread` migration/idempotence fixture described in Step 9 now, before `MigrateLegacyStochasticSpread` exists.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because lossless scalar editing and JSONC ownership behavior are absent.

□ Step 8: Implement a trivia-preserving JSONC syntax document

```
public sealed class JsoncSyntaxDocument
{
    public string OriginalText { get; }
    public bool RequiresOwnedWrite { get; }

    public string ReplaceScalar(string jsonPath, string canonicalJsonToken);
    public string MigrateLegacyStochasticSpread();
}
```

Tokenize strings, punctuation, whitespace, line comments, and block comments without discarding byte spans. Edits replace only the targeted token span. For the historical root `StochasticCDSspread`, move its complete value/trivia span into the existing `CD` object under `StochasticSpread`, remove only the legacy property and necessary comma, and preserve all other source bytes. If both legacy and canonical keys exist, canonical wins and the legacy key is retained with a conflict diagnostic rather than guessing.

□ Step 9: Prove legacy migration and comment preservation

Use a full fixture with a root `StochasticCDSspread` object, an existing `CD` object, comments inside the stochastic object, unknown root/`CD` properties, and CRLF endings. Parse the migrated text again and assert one canonical `CD.StochasticSpread`, no legacy key, unchanged comments/unknown properties, and byte-identical output on a second migration.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 10: Define the derived economic activation boundary

```
public interface IEconomicConfigurationGate
{
    BankResult<ValidatedEconomicConfiguration> Validate(
        BankConfiguration candidate,
        double carriedShortRate);
}
```

Phase 1 may parse and stage an economic candidate but does not activate it for pricing. The financial-core plan supplies the curve/payoff implementation that checks every tenor, shock extreme, liquidity bound, `r=theta`, and carried rate before advancing the configuration revision. The canonical defaults remain the foundation state until that gate succeeds.

Expose `IBankConfigurationStore.Load` and `SaveOwnedEdits` from core so the server host depends on a narrow file-operation port rather than a Vintage Story or filesystem implementation.

□ Step 11: Add the canonical commented template and commit

Copy the complete section 18.1 JSONC block into `firstgearbank.jsonc.template`, embed it in the adapter assembly, and assert in `ConfigurationDefaultsTests` that parsing it equals `BankConfigurationDefaults.Value` with no diagnostic.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
git diff --check
git add FirstGearBank.Core/Configuration "First Gear Bank/Configuration" `
  "First Gear Bank/First Gear Bank.csproj" tests/FirstGearBank.Core.Tests/Configuration
git commit -m "feat: add lossless validated bank configuration"
```

Task 7: Add the Typed, Independently Balanced Journal

Files:

- Create: `FirstGearBank.Core/Ledger/LedgerAccountKind.cs`
- Create: `FirstGearBank.Core/Ledger/LedgerAccountId.cs`
- Create: `FirstGearBank.Core/Ledger/TransactionType.cs`
- Create: `FirstGearBank.Core/Ledger/Posting.cs`
- Create: `FirstGearBank.Core/Ledger/JournalPartySnapshot.cs`
- Create: `FirstGearBank.Core/Ledger/IJournalPayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/PhysicalDepositPayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/PhysicalWithdrawalPayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/SavingsInterestPayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/TemporalGrossInterestPayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/TemporalStoragePayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/RustyTransferPayload.cs`
- Create: `FirstGearBank.Core/Ledger/Payloads/AdminCorrectionPayload.cs`
- Create: `FirstGearBank.Core/Ledger/JournalDraft.cs`
- Create: `FirstGearBank.Core/Ledger/JournalRecord.cs`
- Create: `FirstGearBank.Core/Ledger/JournalRecordFactory.cs`
- Create: `FirstGearBank.Core/Ledger/JournalCanonicalEncoder.cs`
- Create: `FirstGearBank.Core/Ledger/JournalValidationAnchor.cs`
- Create: `FirstGearBank.Core/Ledger/JournalBatchValidator.cs`
- Create: `tests/FirstGearBank.Core.Tests/Ledger/LedgerAccountIdTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Ledger/JournalBalanceTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Ledger/JournalSequenceTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Ledger/JournalHashTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Ledger/JournalPayloadTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/JournalTestData.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/PlayerIds.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/FinancialTimestamps.cs`

Interfaces:

- Consumes: currencies, units, identifiers, timestamps, and failures from Task 5.
- Produces: immutable `JournalDraft/JournalRecord` values and `JournalBatchValidator.Validate`, consumed by projection, coordinator, and persistence.

□ Step 1: Write failing account-identity and per-currency balance tests

```
[Fact]
public void Rusty_and_temporal_postings_may_not_net_against_each_other()
{
    var record = JournalTestData.Record(
        JournalTestData.Posting(
            LedgerAccountId.RustyPhysicalCustody,
            BankCurrency.Rusty,
            new BankUnits(1_000_000)),
        JournalTestData.Posting(
            LedgerAccountId.TemporalPhysicalCustody,
            BankCurrency.Temporal,
```

```

        new BankUnits(-1_000_000)));

var result = JournalBatchValidator.Validate([record], JournalValidationAnchor.Empty);

Assert.Equal(BankErrorCode.UnbalancedJournal, result.Failure?.Code);
}

[Fact]
public void Sum_uses_wider_arithmetic_instead_of_overflowing_long()
{
    var record = JournalTestData.Record(
        JournalTestData.Posting(
            LedgerAccountId.RustyPhysicalCustody,
            BankCurrency.Rusty,
            new BankUnits(long.MaxValue)),
        JournalTestData.Posting(
            LedgerAccountId.RustySystemCorrection,
            BankCurrency.Rusty,
            new BankUnits(long.MaxValue)),
        JournalTestData.Posting(
            LedgerAccountId.RustyInterestExpense,
            BankCurrency.Rusty,
            new BankUnits(2)),
        JournalTestData.Posting(
            LedgerAccountId.RustyCapOverflow,
            BankCurrency.Rusty,
            new BankUnits(long.MinValue)),
        JournalTestData.Posting(
            LedgerAccountId.RustyCapOverflow,
            BankCurrency.Rusty,
            new BankUnits(long.MinValue)));

    Assert.True(JournalBatchValidator.Validate([record], JournalValidationAnchor.Empty).IsSuccess);
}

```

Add tests that customer accounts require a `PlayerId`, CD liability requires a `CdId`, system accounts reject owners, a posting currency must match the account currency, and zero postings are rejected rather than serialized.

□ Step 2: Run the balance tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because ledger accounts, postings, and the validator do not exist.

□ Step 3: Implement stable ledger identities and overflow-safe balancing

Freeze these numeric discriminators:

```

public enum TransactionType : byte
{
    PhysicalDeposit = 1,
    PhysicalWithdrawal = 2,
    SavingsInterest = 3,
    TemporalGrossInterest = 4,
    TemporalStorage = 5,
    CdPurchase = 6,
    CdMaturity = 7,
    RustyTransfer = 8,
    AdminCorrection = 9
}

public enum LedgerAccountKind : byte
{
    CustomerRustySavingsLiability = 1,
    CustomerTemporalVaultLiability = 2,
    RustyPhysicalCustody = 3,
    TemporalPhysicalCustody = 4,
    RustyInterestExpense = 5,
    TemporalInterestExpense = 6,
    TemporalStorageIncome = 7,
    RustyCdPrincipalLiability = 8,
    RustyCdInterestExpense = 9,
    RustyCapOverflow = 10,
    RustySystemCorrection = 11,
}

```

```
TemporalSystemCorrection = 12
}
```

LedgerAccountId has validated factories for player, CD, and system accounts. Posting construction is validated through a factory so no public path creates zero, mismatched-currency, or malformed-owner postings. Sum signed units into `Int128` per currency and require each sum to equal zero.

□ Step 4: Verify balance and account tests pass

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 5: Write failing sequence, uniqueness, payload, and hash-chain tests

```
[Fact]
public void One_bad_record_rejects_the_entire_correlated_batch()
{
    var first = JournalTestData.BalancedRecord(sequence: 7);
    var second = JournalTestData.UnbalancedRecord(sequence: 8);

    var result = JournalBatchValidator.Validate(
        [first, second],
        new JournalValidationAnchor(6, JournalTestData.PreviousHash));

    Assert.Equal(BankErrorCode.UnbalancedJournal, result.Failure?.Code);
}

[Fact]
public void Changing_a_frozen_name_snapshot_breaks_the_hash_chain()
{
    var record = JournalTestData.TransferRecord();
    var changedPayload = ((RustyTransferPayload)record.Payload) with { SenderDisplayName = "Changed" };
    var tampered = JournalTestData.CloneWithPayload(record, changedPayload);

    Assert.Equal(
        BankErrorCode.JournalIntegrityFailure,
        JournalBatchValidator.Validate([tampered], JournalValidationAnchor.Empty).Failure?.Code);
}
```

Add sequence-origin/gap/duplicate tests, duplicate operation ID, mismatched shared `CommandId`, invalid payload/type pairing, overlong name/reason, missing or malformed invocation key, request-key use on an automatic-only transaction, automatic-key use on a player-only transaction, prior-hash mismatch, deterministic canonical bytes, and changed posting/party/timestamp/invocation-key detection. Freeze the admin-correction `NoticeId` derivation with a golden vector and prove a different operation ID or notice domain cannot collide with it.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL on sequence, payload, or canonical-integrity behavior that has not been implemented.

□ Step 6: Implement immutable records and typed payload codecs

```
public interface IJournalPayload
{
    TransactionType TransactionType { get; }
    ushort PayloadSchemaVersion { get; }
}

public sealed class JournalDraft
{
    public OperationId OperationId { get; }
    public BankInvocationIntent InvocationIntent { get; }
    public ushort RecordSchemaVersion { get; }
    public FinancialTimestamp Timestamp { get; }
    public ImmutableArray<JournalPartySnapshot> Parties { get; }
    public IJournalPayload Payload { get; }
    public ImmutableArray<Posting> Postings { get; }

    internal JournalDraft(
        OperationId operationId,
        BankInvocationIntent invocationIntent,
        ushort recordSchemaVersion,
        FinancialTimestamp timestamp,
        ImmutableArray<JournalPartySnapshot> parties,
```

```

        IJournalPayload payload,
        ImmutableArray<Posting> postings);
}

public sealed class JournalRecord
{
    public long Sequence { get; }
    public OperationId OperationId { get; }
    public CommandId CommandId { get; }
    public BankInvocationKey InvocationKey { get; }
    public ushort RecordSchemaVersion { get; }
    public FinancialTimestamp Timestamp { get; }
    public ImmutableArray<JournalPartySnapshot> Parties { get; }
    public IJournalPayload Payload { get; }
    public ImmutableArray<Posting> Postings { get; }
    public ImmutableArray<byte> PreviousRecordHash { get; }
    public ImmutableArray<byte> RecordHash { get; }

    internal JournalRecord(
        long sequence,
        OperationId operationId,
        CommandId commandId,
        BankInvocationKey invocationKey,
        ushort recordSchemaVersion,
        FinancialTimestamp timestamp,
        ImmutableArray<JournalPartySnapshot> parties,
        IJournalPayload payload,
        ImmutableArray<Posting> postings,
        ImmutableArray<byte> previousRecordHash,
        ImmutableArray<byte> recordHash)
    {
        Sequence = sequence;
        OperationId = operationId;
        CommandId = commandId;
        InvocationKey = invocationKey;
        RecordSchemaVersion = recordSchemaVersion;
        Timestamp = timestamp;
        Parties = parties;
        Payload = payload;
        Postings = postings;
        PreviousRecordHash = previousRecordHash;
        RecordHash = recordHash;
    }
}

```

Use sealed payload records, never an authoritative free-form dictionary. Each payload contains exact units and identifiers needed for replay: settlement ID for physical operations; opening/closing/cap flag for accrual; stable sender/recipient IDs plus frozen names and notice ID for transfers; administrator/target IDs plus frozen names, a stable `NoticeId`, mandatory bounded reason, and optional original operation for corrections. Derive a correction notice ID with the versioned domain `FGB/ADMIN-CORRECTION-NOTICE/1\0` from its operation ID, and freeze that derivation with a golden test. CD payload discriminators remain reserved; the certificates plan adds their fully typed contractual records through a section/journal schema migration before any CD can be issued.

Canonical encoding is length-prefixed binary with explicit enum widths and the domain prefix `FGB/JOURNAL/1\0`. Hash prefix `|| previousHash || canonicalRecordWithoutRecordHash` with SHA-256. Use constant-time byte comparison when verifying hashes.

Every new record has exactly one `BankInvocationKey`. The complete pre-finalization `JournalDraft` shape is the one above: it owns its already-reserved `OperationId`, intent, schema, timestamp, parties, typed payload, and postings, and deliberately has no sequence, command ID, resolved invocation key, previous hash, record hash, or revision. Its closed `BankInvocationIntent` union is `Caller` or a particular automatic key plus its canonical payload digest; it has no public way to install a final invocation key. The coordinator resolves `caller` to the admitted request/automatic context and admits every subordinate automatic intent before constructing records. Physical deposit/withdrawal, transfer, and direct correction records require the authenticated caller variant; clock accrual and later CD maturity records require an automatic variant. A multi-record catch-up batch shares the coordinator's one command ID but may contain deterministic subordinate automatic keys only when the coordinator persists all of them in the same candidate; no record may carry an unindexed automatic key.

Every production draft factory requires the exact `OperationId` reserved from the current coordinator-owned mutation identifier scope; factories never mint one and the finalizer never replaces one. `AdminCorrectionPayload.Create` derives

its `NoticeId` from that same operation ID, and `JournalRecordFactory.AdminCorrection` rejects a payload whose derived notice does not match the draft/header operation. The golden notice-vector test therefore covers the actual persisted record identity rather than an unrelated test input.

Create `JournalTestData`, `PlayerIds`, and `FinancialTimestamps` as deterministic test-only builders in the listed support files. They must call production factories for valid records; only explicitly corrupt fixtures may bypass a production factory through an internal test-visible constructor.

□ Step 7: Write and observe failing exact posting-template tests

Add one test per foundation transaction type. For example, a rusty deposit draft must contain exactly custody `+A` and the same player's rusty liability `-A`, in canonical posting order, with no zero entry:

```
[Fact]
public void Physical_deposit_uses_the_locked_two_posting_template()
{
    var draft = JournalRecordFactory.PhysicalDeposit(
        JournalTestData.OperationOne,
        BankInvocationIntent.Caller,
        FinancialTimestamps.MonthFive,
        JournalTestData.RustyDepositPayload(PlayerIds.Ada, units: 250_000));

    Assert.Equal(JournalTestData.OperationOne, draft.OperationId);
    Assert.Equal(2, draft.Postings.Length);
    Assert.Equal(LedgerAccountId.RustyPhysicalCustody, draft.Postings[0].AccountId);
    Assert.Equal(250_000, draft.Postings[0].SignedUnits.Value);
    Assert.Equal(LedgerAccountId.CustomerRustySavings(PlayerIds.Ada), draft.Postings[1].AccountId);
    Assert.Equal(-250_000, draft.Postings[1].SignedUnits.Value);
}
```

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the posting-template factories are not implemented.

□ Step 8: Implement the exact posting templates already fixed by the specification

```
public static class JournalRecordFactory
{
    public static JournalDraft PhysicalDeposit(
        OperationId operationId,
        BankInvocationIntent invocationIntent,
        FinancialTimestamp timestamp,
        PhysicalDepositPayload payload);

    public static JournalDraft PhysicalWithdrawal(
        OperationId operationId,
        BankInvocationIntent invocationIntent,
        FinancialTimestamp timestamp,
        PhysicalWithdrawalPayload payload);

    public static JournalDraft RustyTransfer(
        OperationId operationId,
        BankInvocationIntent invocationIntent,
        FinancialTimestamp timestamp,
        RustyTransferPayload payload);

    public static JournalDraft AdminCorrection(
        OperationId operationId,
        BankInvocationIntent invocationIntent,
        FinancialTimestamp timestamp,
        AdminCorrectionPayload payload);
}
```

Deposit posts custody `+A` and customer liability `-A`; withdrawal reverses both; transfer posts sender liability `+A` and recipient liability `-A`; correction posts the per-currency system correction account `+D` and target liability `-D`. Accrual factories accept already rounded exact components and emit the section 6/9 posting pattern without recalculating economic values. Add an exact correction-factory test proving the draft operation ID, final record header operation ID, and `NoticeId`. `ForAdminCorrection(operationId)` all originate from the same reserved ID.

□ Step 9: Verify all journal tests and commit

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
git diff --check
git add FirstGearBank.Core/Ledger tests/FirstGearBank.Core.Tests/Ledger `
  tests/FirstGearBank.Core.Tests/TestSupport/JournalTestData.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/PlayerIds.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/FinancialTimestamps.cs
git commit -m "feat: add typed balanced banking journal"
```

Task 8: Rebuild Account and System Projections from the Journal

Files:

- Create: `FirstGearBank.Core/Projection/AccountTotals.cs`
- Create: `FirstGearBank.Core/Projection/AccountHistoryDirection.cs`
- Create: `FirstGearBank.Core/Projection/AccountHistoryEntry.cs`
- Create: `FirstGearBank.Core/Projection/PlayerAccountProjection.cs`
- Create: `FirstGearBank.Core/Projection/SystemLedgerProjection.cs`
- Create: `FirstGearBank.Core/Projection/BankProjection.cs`
- Create: `FirstGearBank.Core/Projection/ProjectionCheckpoint.cs`
- Create: `FirstGearBank.Core/Projection/IJournalProjectionRule.cs`
- Create: `FirstGearBank.Core/Projection/ProjectionRuleSet.cs`
- Create: `FirstGearBank.Core/Projection/JournalProjector.cs`
- Create: `FirstGearBank.Core/Projection/ProjectionValidator.cs`
- Create: `tests/FirstGearBank.Core.Tests/Projection/AccountLifecycleProjectionTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Projection/JournalProjectorTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Projection/ProjectionInvariantTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Projection/TransferHistoryProjectionTests.cs`

Interfaces:

- Consumes: a verified contiguous journal and registered typed payload rules from Task 7.
- Produces: `BankProjection`, deterministic replay/checkpoints, and incremental affected-account validation for coordinator and persistence.

□ Step 1: Write failing lazy-account-lifecycle tests

```
[Fact]
public void Successful_physical_deposit_creates_the_first_account_projection()
{
    var record = JournalTestData.PhysicalDeposit(player: PlayerIds.Ada, units: 1_000_000);

    var result = JournalProjector.Foundation.Replay([record], ProjectionCheckpoint.Empty);

    Assert.True(result.IsSuccess);
    Assert.Equal(1_000_000, result.Value.Accounts[PlayerIds.Ada].RustySavings.Value);
}

[Fact]
public void Incoming_transfer_creates_an_offline_recipient_account()
{
    var records = JournalTestData.FinalizeBatch(
        JournalTestData.PhysicalDepositDraft(player: PlayerIds.Ada, units: 2_000_000),
        JournalTestData.RustyTransferDraft(
            sender: PlayerIds.Ada,
            recipient: PlayerIds.Byron,
            units: 500_000));

    var result = JournalProjector.Foundation.Replay(records, ProjectionCheckpoint.Empty);

    Assert.Equal(500_000, result.Value.Accounts[PlayerIds.Byron].RustySavings.Value);
}

[Fact]
public void Administrative_correction_cannot_open_an_account()
{
    var record = JournalTestData.AdminCorrection(player: PlayerIds.Ada, displayedDeltaUnits: 1_000_000);

    Assert.Equal(
        BankErrorCode.AccountNotFound,
```

```

        JournalProjector.Foundation.Replay([record], ProjectionCheckpoint.Empty).Failure?.Code);
    }

```

The final case follows section 5's exclusive rule: only a successful deposit or received transfer opens the relationship. `FinalizeBatch` is the Task 7 deterministic test finalizer: it assigns the two records one contiguous sequence/hash chain from `JournalValidationAnchor.Empty`. It does not concatenate independently finalized records or keep hidden global state.

□ Step 2: Run lifecycle tests and observe missing projection failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because projection types do not exist.

□ Step 3: Implement immutable player/system projection records and rule registration

```

public sealed record PlayerAccountProjection(
    PlayerId PlayerId,
    BankUnits RustySavings,
    BankUnits TemporalVault,
    AccountTotals Totals,
    ImmutableArray<AccountHistoryEntry> History);

public sealed record BankProjection(
    ImmutableDictionary<PlayerId, PlayerAccountProjection> Accounts,
    SystemLedgerProjection SystemLedger,
    long AppliedJournalSequence,
    ImmutableArray<byte> AppliedJournalHash);

public interface IJournalProjectionRule
{
    TransactionType TransactionType { get; }
    BankResult<BankProjection> Apply(BankProjection current, JournalRecord record);
}

```

`ProjectionRuleSet.Foundation` registers physical deposit/withdrawal, both accrual components, transfer, and correction. Duplicate or missing transaction rules fail startup validation. CD rules are added only with their fully typed payloads and schema migration in the certificates plan.

□ Step 4: Implement liability-sign conversion and account-opening rules

Compute displayed customer balances as the checked negation of cumulative customer-liability ledger signs. Deposit may open its subject account. Transfer requires an existing sender and may open only its recipient. Withdrawal, interest, storage, and correction require an existing target. Apply one record to a temporary immutable builder and publish it only after all affected customer balances remain within `[0, MaximumCustomerBalanceUnits]`.

□ Step 5: Verify account lifecycle tests pass

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 6: Write failing replay, aggregate, and directional-history tests

```

[Fact]
public void Complete_replay_matches_incremental_application()
{
    var records = JournalTestData.MixedFoundationHistory();
    var projector = JournalProjector.Foundation;
    var incremental = ProjectionCheckpoint.Empty;

    foreach (var record in records)
    {
        incremental = projector.Apply(incremental, record).Value;
    }

    var replayed = projector.Replay(records, ProjectionCheckpoint.Empty).Value;

    var match = ProjectionValidator.MatchesReplay(incremental.Projection, replayed);

    Assert.True(match.IsSuccess);
    Assert.True(match.Value);
}

```

[Fact]

```

public void One_transfer_has_two_directional_account_views()
{
    var projection = JournalProjector.Foundation
        .Replay(JournalTestData.FundedTransferHistory(), ProjectionCheckpoint.Empty)
        .Value;

    Assert.Equal(AccountHistoryDirection.TransferOut, projection.Accounts[PlayerIds.Ada].History[^1].Direction);
    Assert.Equal(AccountHistoryDirection.TransferIn, projection.Accounts[PlayerIds.Byron].History[^1].Direction);
    Assert.Equal(
        projection.Accounts[PlayerIds.Ada].History[^1].OperationId,
        projection.Accounts[PlayerIds.Byron].History[^1].OperationId);
}

```

Add cumulative deposit/withdrawal/interest/storage/transfer/correction totals, signed negative system accounts, history order, unknown payload rule, checkpoint hash mismatch, journal gap, negative customer result, cap breach, and corrupted cached projection comparison.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because complete replay, history, or cache comparison is not implemented.

□ Step 7: Implement replay, checkpoints, totals, and cache comparison

```

public sealed class JournalProjector
{
    public static JournalProjector Foundation { get; }

    public BankResult<ProjectionCheckpoint> Apply(
        ProjectionCheckpoint checkpoint,
        JournalRecord nextRecord);

    public BankResult<BankProjection> Replay(
        IEnumerable<JournalRecord> records,
        ProjectionCheckpoint checkpoint);
}

public static class ProjectionValidator
{
    public static BankResult<bool> MatchesReplay(
        BankProjection cached,
        BankProjection replayed);
}

```

Replay first validates journal continuity/hash/balance, then applies one typed rule at a time. Incremental application touches only parties and ledger accounts named by the record. Cache mismatch returns a repairable diagnostic; it never creates a journal correction or promotes the cache over replay.

□ Step 8: Verify all projection tests and commit

```

dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
git diff --check
git add FirstGearBank.Core/Projection tests/FirstGearBank.Core.Tests/Projection
git commit -m "feat: add replayable bank projections"

```

Task 9: Serialize Mutations Through the Idempotent Banking Coordinator

Files:

- Create: FirstGearBank.Core/State/BankState.cs
- Create: FirstGearBank.Core/State/BankStateSnapshot.cs
- Create: FirstGearBank.Core/State/BankStateRepository.cs
- Create: FirstGearBank.Core/State/BankStateValidator.cs
- Create: FirstGearBank.Core/State/BankDomainAvailability.cs
- Create: FirstGearBank.Core/State/BankDomainSlot.cs
- Create: FirstGearBank.Core/State/FinanceDomainState.cs
- Create: FirstGearBank.Core/State/RequestControlState.cs
- Create: FirstGearBank.Core/Recovery/QuarantineDomain.cs
- Create: FirstGearBank.Core/Recovery/BankQuarantineState.cs
- Create: FirstGearBank.Core/Recovery/QuarantineTransition.cs

- Create: `FirstGearBank.Core/Recovery/AuthorizedRecoveryContext.cs`
- Create: `FirstGearBank.Core/Recovery/ArchivedRecoveryEvidence.cs`
- Create: `FirstGearBank.Core/Recovery/RecoveryAuditEntry.cs`
- Create: `FirstGearBank.Core/Recovery/RecoveryControlState.cs`
- Create: `FirstGearBank.Core/Recovery/RecoveryLimits.cs`
- Create: `FirstGearBank.Core/Requests/TerminalResponse.cs`
- Create: `FirstGearBank.Core/Requests/ProcessedRequest.cs`
- Create: `FirstGearBank.Core/Requests/OperationScope.cs`
- Create: `FirstGearBank.Core/Requests/RequestRegistry.cs`
- Create: `FirstGearBank.Core/Requests/TerminalResponseCache.cs`
- Create: `FirstGearBank.Core/Requests/ProcessedAutomaticOperation.cs`
- Create: `FirstGearBank.Core/Requests/AutomaticOperationRegistry.cs`
- Create: `FirstGearBank.Core/Coordination/AuthenticatedCommandContext.cs`
- Create: `FirstGearBank.Core/Coordination/AutomaticCommandContext.cs`
- Create: `FirstGearBank.Core/Coordination/IBankMutation.cs`
- Create: `FirstGearBank.Core/Coordination/IBankRecoveryMutation.cs`
- Create: `FirstGearBank.Core/Coordination/IBankControlChange.cs`
- Create: `FirstGearBank.Core/Coordination/BankControlChangeRuleSet.cs`
- Create: `FirstGearBank.Core/Coordination/BankMutationReadView.cs`
- Create: `FirstGearBank.Core/Coordination/MutationDisposition.cs`
- Create: `FirstGearBank.Core/Coordination/MutationDecision.cs`
- Create: `FirstGearBank.Core/Coordination/CoordinatorResponse.cs`
- Create: `FirstGearBank.Core/Coordination/IFinancialTimestampSource.cs`
- Create: `FirstGearBank.Core/Coordination/IIdentifierSource.cs`
- Create: `FirstGearBank.Core/Coordination/IMutationIdentifierScope.cs`
- Create: `FirstGearBank.Core/Coordination/BankingCoordinator.cs`
- Create: `tests/FirstGearBank.Core.Tests/Requests/CommandPayloadDigestAdmissionTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Requests/RequestRegistryTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Requests/AutomaticOperationRegistryTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Coordination/BankingCoordinatorTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Recovery/QuarantineTransitionTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/State/BankStateRepositoryTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/RequestTestData.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/CoordinatorTestRig.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/CoordinatorTestMutations.cs`

Interfaces:

- Consumes: immutable journal/projection contracts and the revision vector.
- Produces: authenticated and automatic `BankingCoordinator.Execute` paths, immutable `BankStateSnapshot`, staging-state semantics, and replay barriers used by all server mutations and persistence.

□ Step 1: Write failing canonical-digest and request-admission tests

```
[Fact]
public void Digest_is_domain_and_schema_separated()
{
    var payload = new byte[] { 10, 20, 30 };

    var v1 = CommandPayloadDigest.Compute("deposit", 1, payload);
    var v2 = CommandPayloadDigest.Compute("deposit", 2, payload);
    var other = CommandPayloadDigest.Compute("withdraw", 1, payload);

    Assert.False(v1.FixedTimeEquals(v2));
    Assert.False(v1.FixedTimeEquals(other));
}

[Fact]
public void Accepted_business_rejection_advances_high_water_without_a_journal_record()
{
    var registry = RequestRegistry.Empty.Open(RequestTestData.Scope);

    var result = registry.FinalizeNext(
        RequestTestData.Key(sequence: 1),
        RequestTestData.Digest,
```

```

        TerminalResponse.Rejected(BankErrorCode.InsufficientFunds),
        committedOperationIds: []);

Assert.Equal(1, result.Value.Scopes[RequestTestData.Scope.Nonce].HighWaterSequence);
Assert.Empty(result.Value.PermanentCommittedKeys);
}

```

Add inactive scope, sequence zero, sequence gap, identical cached retry, changed-payload reuse, cache eviction, expired response, successful permanent key, close, and restart invalidation cases. Add automatic-operation cases proving that a successful deterministic key is permanent, an identical duplicate returns its stored result without evaluation, a changed operation cannot reuse the key, and save/reload preserves the barrier.

□ Step 2: Run request tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because request-state types do not exist.

□ Step 3: Implement versioned digests, scopes, caches, and replay barriers

Compute:

```

SHA-256(
  ASCII "FGB/CMD-PAYLOAD/1\0" ||
  UInt16BigEndian(command-kind UTF-8 byte length) || command-kind bytes ||
  UInt16BigEndian(command schema version) ||
  UInt32BigEndian(payload length) || canonical payload bytes)

```

Use constant-time digest comparison. `OperationScope` binds one authenticated `PlayerId`, a nonreused nonce, the next sequence, and bounded terminal responses. Production keeps at least 1,024 full terminal bodies per player; constructors accept a lower positive limit for tests. Only successful financial requests enter `PermanentCommittedKeys`. A processed sequence whose body was pruned returns `AlreadyProcessedResponseExpired`; it never executes again.

`CommandPayloadDigest` is a 32-byte value type with structural `Equals/GetHashCode`, a `FixedTimeEquals` method used for admission, and a defensive-copy byte export. It never inherits the shallow equality behavior of an immutable array.

`AutomaticOperationRegistry` permanently maps each `AutomaticOperationKey` to its committed operation IDs and resulting revision together with the domain/schema-separated digest of its canonical automatic-command payload. Successful automatic financial work, including a deliberate rounded-zero checkpoint, installs that mapping in the same revision. An identical duplicate returns the stored result before capturing time or issuing identifiers; the same key with a different digest returns `RequestPayloadMismatch`. An invariant fault does not install the key and may be retried after the fault is repaired.

□ Step 4: Verify all request-state tests pass

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 5: Write failing coordinator atomicity and one-timestamp tests

```

[Fact]
public void Correlated_records_publish_together_with_one_timestamp()
{
    var clock = new CountingTimestampSource(FinancialTimestamps.MonthFive);
    var coordinator = CoordinatorTestRig.Create(clock);
    var mutation = new TwoRecordMutation(PlayerIds.Byron, PlayerIds.Ada);

    var response = coordinator.Execute(RequestTestData.Context(sequence: 1), mutation);
    var finance = coordinator.Current.Finance.TryGetVerified().Value;

    Assert.True(response.IsSuccess);
    Assert.Equal(1, clock.CaptureCount);
    Assert.Equal(2, finance.Journal.Length);
    Assert.All(
        finance.Journal,
        record => Assert.Equal(FinancialTimestamps.MonthFive, record.Timestamp));
}

[Fact]
public void Invalid_second_record_publishes_neither_record()
{
    var coordinator = CoordinatorTestRig.Create();
}

```

```

var before = coordinator.Current;

var response = coordinator.Execute(
    RequestTestData.Context(sequence: 1),
    new MutationWithUnbalancedSecondRecord());
var currentFinance = coordinator.Current.Finance.TryGetVerified().Value;
var currentRequests = coordinator.Current.RequestControl.TryGetVerified().Value;

Assert.Equal(BankErrorCode.UnbalancedJournal, response.Failure?.Code);
Assert.True(before.Finance.TryGetVerified().Value.Journal.AsSpan().SequenceEqual(currentFinance.Journal.AsSpan()));
Assert.Same(before.Finance.TryGetVerified().Value.Projection, currentFinance.Projection);
Assert.Equal(before.Revisions.Global.Value + 1, coordinator.Current.Revisions.Global.Value);
Assert.Equal(1, currentRequests.Requests.GetHighWater(RequestTestData.Scope.Nonce));
}

[Fact]
public void Mutation_cannot_emit_finance_while_declaring_only_configuration()
{
    var coordinator = CoordinatorTestRig.Create();

    var response = coordinator.Execute(
        RequestTestData.Context(sequence: 1),
        new UnderDeclaredFinanceMutation());

    Assert.Equal(BankErrorCode.UndeclaredMutationDomain, response.Failure?.Code);
    Assert.Empty(coordinator.Current.Finance.TryGetVerified().Value.Journal);
    Assert.Equal(
        1,
        coordinator.Current.RequestControl.TryGetVerified().Value.Requests.GetHighWater(RequestTestData.Scope.Nonce));
}

[Fact]
public void Undeclared_quarantined_read_is_blocked_even_when_the_mutation_ignores_the_failure()
{
    var coordinator = CoordinatorTestRig.WithUnavailableFinance();

    var response = coordinator.Execute(
        RequestTestData.Context(sequence: 1),
        new ConfigurationMutationThatAttemptsFinanceRead());

    Assert.Equal(BankErrorCode.UndeclaredMutationDomain, response.Failure?.Code);
    Assert.True(coordinator.Current.Finance.IsUnavailable);
    Assert.False(coordinator.Current.RequestControl.TryGetVerified().Value.Requests.HasSuccessful(RequestTestData.Key(1)));
}

```

Both adversarial fixtures declare only `Configuration`: the first emits a finance draft, and the second calls `ReadFinance`, ignores its failure, and otherwise returns a configuration-only decision. Add tests for ordinal `PlayerId` participant ordering, duplicate operation IDs, cap/negative projection rejection, quarantined required domain, terminal zero-record rejection, replay without a new clock/identifier call, and one global plus only-changed subsystem revision increment. Exercise the automatic overload with a deterministic month-boundary key and prove duplicate and restored duplicate calls do not recapture time or issue a command/operation ID. Add identifier-contract cases for a draft that uses a nonissued operation ID, two drafts that reuse one reserved ID, an accepted decision that leaves a reserved operation ID unused, and final records/terminal replay metadata that preserve the exact draft IDs in canonical draft order. Each case must fail or succeed because of the real coordinator result and state, not because a fake source was merely called. In `QuarantineTransitionTests`, cover undeclared/unavailable reads, ordinary-change attempts to touch quarantine, wrong administrator, missing permission, empty/overlong reason, stale evidence digest, clear without a verified replacement, successful set and clear, exact raw-evidence archival, audit identity/frozen name/reason, changed `Recovery` plus target revisions, and rejection before mutation when the evidence archive would exceed its structural bound. In `BankStateRepositoryTests`, prove capture alone does not clear `HasUnstagedChanges`, exact-revision staging success clears only that flag, `RequiresEngineSaveRestage` remains true, a newer publication cannot be cleared by an older staging result, and staging failure retains both the state and diagnostic.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because atomic coordinator publication and replay behavior are not implemented.

□ Step 6: Implement the mutation and coordinator contracts

```

public interface IBankMutation
{

```

```

// Every business domain the mutation may read or change. Requests is coordinator-owned and implicit.
BankMutationDomains RequiredDomains { get; }
ImmutableArray<PlayerId> Participants { get; }

MutationDecision Evaluate(
    BankMutationReadView current,
    FinancialTimestamp timestamp,
    CommandId commandId,
    IMutationIdentifierScope identifiers);
}

public interface IBankRecoveryMutation : IBankMutation
{
    QuarantineTransition RequestedTransition { get; }
}

public interface IIdentifierSource
{
    CommandId NextCommandId();
    OperationId NextOperationId();
    SettlementId NextSettlementId();
}

public interface IMutationIdentifierScope
{
    OperationId ReserveOperationId();
    SettlementId ReserveSettlementId();
}

public sealed record AuthenticatedCommandContext(
    PlayerId AuthenticatedPlayer,
    RequestKey RequestKey,
    CommandPayloadDigest PayloadDigest);

public enum MutationDisposition : byte
{
    Accepted = 1,
    Rejected = 2
}

public interface IBankControlChange
{
    BankMutationDomains Domain { get; }
    BankInvocationIntent InvocationIntent { get; }
}

public sealed class BankMutationReadView
{
    public BankResult<FinanceDomainState> ReadFinance();
    public BankResult<BankConfiguration> ReadConfiguration();

    internal BankMutationDomains ObservedReadDomains { get; }
}

public sealed class MutationDecision
{
    public MutationDisposition Disposition { get; }
    public BankFailure? Failure { get; }
    public ImmutableArray<JournalDraft> JournalDrafts { get; }
    public ImmutableArray<IBankControlChange> ControlChanges { get; }

    public static MutationDecision Accepted(
        ImmutableArray<JournalDraft> journalDrafts,
        ImmutableArray<IBankControlChange> controlChanges);

    public static MutationDecision Rejected(BankFailure failure);

    private MutationDecision(
        MutationDisposition disposition,
        BankFailure? failure,
        ImmutableArray<JournalDraft> journalDrafts,
        ImmutableArray<IBankControlChange> controlChanges);
}

```

```

public enum BankDomainAvailability : byte
{
    Verified = 1,
    Unavailable = 2
}

public sealed class BankDomainSlot<T>
{
    public BankDomainAvailability Availability { get; }
    public bool IsUnavailable { get; }
    public BankResult<T> TryGetVerified();

    public static BankDomainSlot<T> Verified(T value);
    public static BankDomainSlot<T> Unavailable(QuarantineDomain domain, EvidenceDigest evidenceDigest);
}

public sealed record FinanceDomainState(
    ImmutableArray<JournalRecord> Journal,
    BankProjection Projection);

public sealed record RequestControlState(
    RequestRegistry Requests,
    AutomaticOperationRegistry AutomaticOperations);

public enum QuarantineTransitionKind : byte
{
    Set = 1,
    ClearAfterRepair = 2
}

public sealed record QuarantineTransition(
    QuarantineTransitionKind Kind,
    QuarantineDomain Domain,
    EvidenceDigest ExpectedEvidenceDigest,
    string MandatoryReason);

public sealed class AuthorizedRecoveryContext
{
    public PlayerId Administrator { get; }
    public string FrozenAdministratorName { get; }
    public string GrantedPermission { get; }

    internal AuthorizedRecoveryContext(
        PlayerId administrator,
        string frozenAdministratorName,
        string grantedPermission);
}

public sealed record RecoveryControlState(
    ImmutableDictionary<EvidenceDigest, ArchivedRecoveryEvidence> EvidenceArchive,
    ImmutableArray<RecoveryAuditEntry> AuditEntries);

public static class RecoveryLimits
{
    public const int MaximumArchivedEvidenceBytes = 192 * 1024 * 1024;
}

public sealed record BankState(
    BankDomainSlot<FinanceDomainState> Finance,
    BankDomainSlot<RequestControlState> RequestControl,
    BankDomainSlot<BankConfiguration> Configuration,
    RecoveryControlState RecoveryControl,
    BankQuarantineState Quarantine,
    BankRevisionVector Revisions);

public sealed record AutomaticCommandContext(
    AutomaticOperationKey OperationKey,
    CommandPayloadDigest PayloadDigest);

public sealed class BankingCoordinator
{
    public CoordinatorResponse Execute(
        AuthenticatedCommandContext context,
        IBankMutation mutation);
}

```

```

public CoordinatorResponse Execute(
    AutomaticCommandContext context,
    IBankMutation mutation);

public CoordinatorResponse ExecuteRecovery(
    AuthenticatedCommandContext context,
    AuthorizedRecoveryContext authorization,
    IBankRecoveryMutation mutation);
}

public sealed record CoordinatorResponse(
    BankInvocationKey InvocationKey,
    bool IsSuccess,
    BankFailure? Failure,
    ImmutableArray<OperationId> OperationIds,
    BankRevision ResultingRevision);

```

MutationDecision factories enforce that Accepted has no failure and Rejected has one failure and no drafts/control changes. RequiredDomains is the complete set of business domains that Evaluate may read or change; it excludes the coordinator-owned Requests domain, which every admitted invocation gets implicitly. The mutation never receives BankState. The coordinator constructs a fresh BankMutationReadOnly with only the declared, verified immutable domain slices. Every Read* call records its domain before returning; an undeclared or unavailable read returns DomainUnavailable without exposing a placeholder or underlying value. A mutation that ignores that failure is still rejected afterward because the view records the attempted domain. Later phases add explicit typed accessors as domains become real; no generic escape hatch, service locator, reflection accessor, or raw-state property is permitted.

Only registered sealed control-change types may run. IBankControlChange carries data but has no Apply(BankState) method. BankControlChangeRuleSet owns the closed type switch, reads only the change's one declared non-Finance/non-Requests domain, and applies its typed replacement/delta to a temporary candidate. Explicit domain-specific semantic comparers prove that only the declared top-level domain changed; this task cannot depend on the persistence codecs introduced later. It validates the complete candidate and rejects any declared/actual-domain mismatch.

Before evaluation, compute admittedDomains = mutation.RequiredDomains | BankMutationDomains.Requests, reject unknown bits, and prove none of those domains is quarantined or unavailable. After evaluation and typed control-change application, independently derive the intrinsic business-domain union: include Finance when any journal draft exists, then union every control change's single declared domain. Reject with UndeclaredMutationDomain unless both the view's observed-read domains and that intrinsic write union are subsets of mutation.RequiredDomains. Immediately before publication, re-run the availability/quarantine check against the observed-read and intrinsic-write union plus Requests on the fully validated candidate. This second check is mandatory defense in depth even though publication remains under one lock.

BankDomainSlot<T> represents verified authority or explicit unavailability; it never manufactures an empty journal, registry, configuration, or other authoritative value after corruption. TryGetVerified is the only production value access. BankStateValidator requires every unavailable slot to have a matching quarantine entry and evidence digest and every verified slot to satisfy its domain validator. This lets an unrelated verified domain advance while opaque corrupt bytes remain outside the slot for exact persistence pass-through.

Ordinary mutations and control changes cannot alter BankQuarantineState or RecoveryControlState. Quarantine set/clear is available only through ExecuteRecovery, an authenticated permission-checked AuthorizedRecoveryContext, and a registered sealed IBankRecoveryMutation. The coordinator proves the administrator matches the authenticated caller, the exact permission is granted, the mandatory reason is bounded, and ExpectedEvidenceDigest matches the current entry. A clear transition must install a fully validated repaired domain value and move the byte-identical displaced raw evidence into the authoritative RecoveryControlState archive before removing the block; a set transition adds evidence and the block atomically. Both append an immutable audit entry and advance the target plus Recovery revisions. Stale evidence, unregistered recovery types, missing permission, archive-size overflow, or any attempt to clear without replacement rejects. If retaining both repaired state and evidence would exceed persistence bounds, clearing remains blocked until a later permission-gated export/prune workflow is explicitly designed; evidence is never silently discarded. AuthorizedRecoveryContext has an internal constructor visible only to the trusted server adapter and core tests. The adapter constructs it only after the engine's permission API accepts the authenticated server player; no network DTO carries or selects GrantedPermission. ExecuteRecovery still requires verified RequestControl and RecoveryControl, but it may admit its one explicitly named unavailable target without exposing a fabricated target value. Its recovery view supplies only the matching quarantine metadata/evidence and the mutation-supplied replacement; all nontarget domain reads remain gated normally. The replacement must pass the target's complete validator and any cross-domain checks before the transition can clear the

slot.

The coordinator obtains its one `CommandId` directly from the injected `IIdentifierSource`, then gives `Evaluate` only a new invocation-scoped tracking `IMutationIdentifierScope`. Every factory receives an operation ID returned by `ReserveOperationId`; an accepted decision must use each reserved operation ID exactly once, may use no nonissued operation ID, and may not duplicate one. Violation returns `IdentifierContractViolation` and publishes no business state. The coordinator finalizer copies each draft's operation ID unchanged into its record header and stores the same IDs, in canonical draft order, in the terminal request or automatic-operation entry and `CoordinatorResponse`. Task 10 extends the tracker: each reserved settlement ID must be consumed by one matching physical payload/control-change pair, and that pair's associated operation-ID array must equal the exact finalized record IDs.

Under one private coordinator lock: pre-admit the authenticated request or automatic root key; sort distinct participants with `StringComparer.Ordinal` over the exact UID value; capture one timestamp and command ID; create the tracking identifier scope; evaluate one immutable decision; validate its identifier and intrinsic-domain declarations; pre-admit every subordinate automatic intent and reject any other request key; resolve invocation intents; finalize sequences/hashes while preserving the reserved operation IDs; replay/validate every new record and affected projection; validate typed control changes; recheck candidate quarantine; advance the revision vector; finalize the request and every automatic replay barrier; then exchange the repository reference once. No service may mutate a projection or journal outside this path.

Authenticated pre-admission first proves that `RequestKey.PlayerId`, its scope owner, and `AuthenticatedPlayer` are the same exact `PlayerId`; a mismatch is preadmission failure and never reaches the mutation.

If an admitted business request rejects, publish one valid zero-journal control revision that advances Requests only. If a candidate has an arithmetic/time/journal invariant fault, discard it and build the terminal authenticated-request control revision from the prior state. Preadmission failures such as inactive scope, wrong sequence, changed digest, or a duplicate automatic key do not advance state. Automatic invariant failures publish nothing; successful automatic decisions install their permanent key atomically, even when their validated financial result has zero journal records.

The listed coordinator support files contain deterministic fake clocks/identifier sources and the two-record, rejecting, and deliberately corrupt `IBankMutation` implementations used by these tests. They do not duplicate coordinator logic.

□ Step 7: Implement immutable snapshot and staging semantics

```
public sealed class BankStateRepository
{
    public bool HasUnstagedChanges { get; }
    public bool RequiresEngineSaveRestage { get; }
    public BankRevision? LastStagedRevision { get; }

    public BankStateSnapshot CaptureForStaging();
    public void Publish(BankState next);
    public void MarkStagingSucceeded(BankRevision snapshottedRevision);
    public void MarkStagingFailed(BankRevision snapshottedRevision, Exception exception);
}
```

`CaptureForStaging` returns the current immutable state and revision without clearing `HasUnstagedChanges`. `MarkStagingSucceeded` clears that flag only when the current global revision still equals the revision accepted by both world-data staging calls. It records `LastStagedRevision` but never calls it durably saved. A separate `RequiresEngineSaveRestage` flag remains true throughout the live host because Vintage Story exposes no post-disk-save acknowledgment; the adapter restages the current snapshot on every `GameWorldSave`. A staging failure or newer publish retains `HasUnstagedChanges` as well.

□ Step 8: Verify coordinator/repository behavior under concurrency

Run 100 parallel test calls with valid independently issued scopes and assert journal sequences are contiguous, operation IDs unique, no lost updates occur, and final projection equals replay. The test uses a barrier to overlap callers but no wall-clock sleeps.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 9: Run all state tests and commit

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
git diff --check
git add FirstGearBank.Core/State FirstGearBank.Core/Recovery FirstGearBank.Core/Requests `
    FirstGearBank.Core/Coordination tests/FirstGearBank.Core.Tests/Requests `
    tests/FirstGearBank.Core.Tests/Coordination tests/FirstGearBank.Core.Tests/Recovery `
    tests/FirstGearBank.Core.Tests/State `
    tests/FirstGearBank.Core.Tests/TestSupport/RequestTestData.cs `
    tests/FirstGearBank.Core.Tests/TestSupport/CoordinatorTestRig.cs `
```

```
tests/FirstGearBank.Core.Tests/TestSupport/CoordinatorTestMutations.cs
git commit -m "feat: add atomic idempotent banking coordinator"
```

Task 10: Freeze the Physical-Settlement Recovery Evidence Contract

Files:

- Create: FirstGearBank.Core/Settlement/PhysicalSettlementDirection.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementPhase.cs
- Create: FirstGearBank.Core/Settlement/SettlementManifestLine.cs
- Create: FirstGearBank.Core/Settlement/SettlementManifest.cs
- Create: FirstGearBank.Core/Settlement/InventoryFingerprint.cs
- Create: FirstGearBank.Core/Settlement/InventorySlotAddress.cs
- Create: FirstGearBank.Core/Settlement/InventorySlotDeltaV1.cs
- Create: FirstGearBank.Core/Settlement/InventoryMutationPlanV1.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementLimits.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementRecoveryPlanV1.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementRecoveryPlanCodec.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementDomainState.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementRecord.cs
- Create: FirstGearBank.Core/Settlement/SettlementRecoveryCapsule.cs
- Create: FirstGearBank.Core/Settlement/RecoveryAuthenticationKey.cs
- Create: FirstGearBank.Core/Settlement/SettlementCapsuleAuthenticator.cs
- Create: FirstGearBank.Core/Settlement/SettlementRecoveryEvidence.cs
- Create: FirstGearBank.Core/Settlement/SettlementRecoveryDecision.cs
- Create: FirstGearBank.Core/Settlement/SettlementReconciler.cs
- Create: FirstGearBank.Core/Settlement/VerifiedPhysicalRecoveryContext.cs
- Create: FirstGearBank.Core/Settlement/PhysicalSettlementControlChange.cs
- Create: FirstGearBank.Core/Settlement/IPlayerRecoveryCapsuleStore.cs
- Create: tests/FirstGearBank.Core.Tests/Settlement/SettlementCapsuleAuthenticatorTests.cs
- Create: tests/FirstGearBank.Core.Tests/Settlement/PhysicalSettlementRecoveryPlanCodecTests.cs
- Create: tests/FirstGearBank.Core.Tests/Settlement/SettlementRecoveryMatrixTests.cs
- Create: tests/FirstGearBank.Core.Tests/Settlement/CapsuleOnlyCoordinatorRecoveryTests.cs
- Create: tests/FirstGearBank.Core.Tests/Settlement/SettlementContractBoundsTests.cs
- Create: tests/FirstGearBank.Core.Tests/TestSupport/SettlementTestData.cs
- Create: tests/FirstGearBank.Core.Tests/TestSupport/Evidence.cs
- Create: tests/FirstGearBank.Core.Tests/TestSupport/TestKeys.cs
- Modify: FirstGearBank.Core/State/BankState.cs:1
- Modify: FirstGearBank.Core/State/BankStateValidator.cs:1
- Modify: FirstGearBank.Core/Requests/RequestRegistry.cs:1
- Modify: FirstGearBank.Core/Coordination/BankingCoordinator.cs:1
- Modify: tests/FirstGearBank.Core.Tests/Coordination/BankingCoordinatorTests.cs:1
- Modify: docs/architecture/0001-physical-settlement-recovery-threat-model.md:1
- Modify: docs/architecture/0001-physical-settlement-recovery-threat-model.tex:1 through regeneration
- Generate but do not commit: docs/architecture/0001-physical-settlement-recovery-threat-model.pdf

Interfaces:

- Consumes: canonical IDs, money, hashes, and revision values from Tasks 5 and 9.
- Produces: the authenticated record/capsule formats and exact reconciliation decision matrix persisted by Task 11 and implemented against inventory in the physical-settlement plan.

□ Step 1: Write failing capsule roundtrip and tamper tests

[Fact]

```
public void Capsule_authenticates_only_for_the_exact_world_player_and_manifest()
{
    using var key = RecoveryAuthenticationKey.FromBytes(TestKeys.ThirtyTwoBytes);
    var capsule = SettlementTestData.UnsignedCapsule();
    var signed = SettlementCapsuleAuthenticator.Sign(key, capsule);

    Assert.True(SettlementCapsuleAuthenticator.Verify(key, signed));
    Assert.False(SettlementCapsuleAuthenticator.Verify(
```

```

    key,
    signed with { Plan = signed.Plan with { PlayerId = PlayerIds.Byron } }));
Assert.False(SettlementCapsuleAuthenticator.Verify(
    key,
    signed with
    {
        Plan = signed.Plan with { Manifest = SettlementTestData.DifferentManifest }
    }));
}

```

Independently tamper every recovery-plan field: world binding, player, settlement ID, request key, request payload digest, command ID, financial/world timestamp, direction, manifest line order/content, before/after fingerprint, inventory address, slot index, before/after stack bytes, draft operation/intent/schema/party/payload/postings, associated operation IDs, terminal response, and phase. Test a wrong key, short/long key, short/long tag, duplicate item lines/slot addresses, overlong asset or inventory code, zero/negative count, invalid stack-codec version, arithmetic overflow, and every exact size boundary.

□ Step 2: Run capsule tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the settlement evidence types do not exist.

□ Step 3: Implement bounded canonical evidence and HMAC authentication

Use only the four persisted phases locked by section 10.1:

```

public enum PhysicalSettlementPhase : byte
{
    Prepared = 1,
    InventoryApplied = 2,
    BankCommitted = 3,
    Finalized = 4
}

public sealed record InventorySlotAddress(
    string InventoryCode,
    int SlotIndex);

public sealed record InventorySlotDeltaV1(
    InventorySlotAddress Address,
    ImmutableArray<byte> CanonicalBeforeStack,
    ImmutableArray<byte> CanonicalAfterStack);

public sealed record InventoryMutationPlanV1(
    ushort StackCodecVersion,
    ImmutableArray<InventorySlotDeltaV1> SlotDeltas);

public static class PhysicalSettlementLimits
{
    public const int MaximumJournalDrafts = 16;
    public const int MaximumInventorySlotDeltas = 64;
    public const int MaximumCanonicalStackBytes = 64 * 1024;
    public const int MaximumRecoveryPlanBytes = 4 * 1024 * 1024;
}

public sealed record PhysicalSettlementRecoveryPlanV1(
    ushort PlanSchemaVersion,
    EvidenceDigest WorldBindingHash,
    PlayerId PlayerId,
    SettlementId SettlementId,
    RequestKey RequestKey,
    CommandPayloadDigest PayloadDigest,
    CommandId CommandId,
    FinancialTimestamp Timestamp,
    PhysicalSettlementDirection Direction,
    SettlementManifest Manifest,
    InventoryFingerprint BeforeStateFingerprint,
    InventoryFingerprint AfterStateFingerprint,
    InventoryMutationPlanV1 InventoryMutation,
    ImmutableArray<JournalDraft> PredeclaredBankDrafts,
    ImmutableArray<OperationId> AssociatedOperationIds,
    TerminalResponse SuccessfulTerminalResponse);

```

```
public sealed record SettlementRecoveryCapsule(
    ushort CapsuleSchemaVersion,
    PhysicalSettlementRecoveryPlanV1 Plan,
    PhysicalSettlementPhase ProvenPhase,
    ImmutableArray<byte> AuthenticationTag);
```

`PhysicalSettlementRecord` stores this same complete recovery plan plus phase; a capsule never points at Bank-only data. `PhysicalSettlementRecoveryPlanCodec` encodes with FGB/PHYSICAL-SETTLEMENT-RECOVERY-PLAN/1\0, explicit numeric widths, length-prefixed UTF-8/bytes, sorted unique slot addresses, and the Task 7 canonical encodings of every complete draft field. Empty stack bytes mean an empty slot. The phase-3 adapter must prove a stable player-inventory address and versioned canonical Bank-item stack codec before it may advertise co-serialization or automatically apply a slot delta; until then capability remains `UnprovedCoSerialization`.

Validate the plan as one closed contract: request/player agree; every draft is `Caller`, shares the exact stored timestamp, uses a physical payload with the same settlement/player/direction/manifest facts, and is independently balanced; its associated operation-ID array equals the draft IDs in canonical order; and its bounded terminal response is successful and describes those exact IDs. The stored command ID and request digest are the original admitted values. Sequence numbers, previous hashes, record hashes, and resulting revision are deliberately absent because the coordinator assigns them against the verified current Bank state during initial commit or recovery.

Hash the canonical draft array with FGB/PHYSICAL-SETTLEMENT-BANK-BATCH/1\0; expose that derived digest for diagnostics and cross-checks, but retain the drafts themselves because a digest is not reconstructive evidence. Canonicalize the outer capsule with FGB/SETTLEMENT-CAPSULE/1\0 over schema, complete plan bytes, and phase, excluding the authentication-tag field itself. Generate a random 32-byte key through the Task 5 `IEntropySource`; production uses `RandomNumberGenerator.Fill`. Never expose it in logs, clear owned key memory on dispose, sign with HMAC-SHA-256, and verify with `CryptographicOperations.FixedTimeEquals`.

Bound a plan to 16 journal drafts, 64 slot deltas, 64 KiB per canonical stack, and 4 MiB total canonical plan bytes. Reject limits before allocating or iterating nested payloads. Golden fixtures independently hand-code one deposit and one withdrawal plan and assert byte-identical plan encoding, batch digest, capsule bytes, and authentication tag.

□ Step 4: Verify capsule and bounds tests pass

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 5: Write the exact failing reconciliation-matrix tests

```
[Theory]
[MemberData(nameof(RecoveryCases))]
public void Recovery_matrix_has_one_conservative_action(
    SettlementRecoveryEvidence evidence,
    SettlementRecoveryAction expected)
{
    Assert.Equal(expected, SettlementReconciler.Decide(evidence).Action);
}

public static TheoryData<SettlementRecoveryEvidence, SettlementRecoveryAction> RecoveryCases => new()
{
    { Evidence.NeitherSide, SettlementRecoveryAction.DiscardOrphanPrepared },
    { Evidence.ProvedCoSerializedCapsuleOnly, SettlementRecoveryAction.AppendPredeclaredBankBatch },
    { Evidence.UnprovedCoSerializedCapsuleOnly, SettlementRecoveryAction.Quarantine },
    { Evidence.BankOnlyFingerprintMatches, SettlementRecoveryAction.ApplyInventoryAndCapsule },
    { Evidence.BothSides, SettlementRecoveryAction.MarkFinalized },
    { Evidence.BankOnlyFingerprintChanged, SettlementRecoveryAction.Quarantine },
    { Evidence.InvalidCapsuleOnly, SettlementRecoveryAction.Quarantine },
    { Evidence.ContradictoryPhases, SettlementRecoveryAction.Quarantine }
};

[Fact]
public void Proved_capsule_only_recovery_commits_the_exact_plan_once_across_restart()
{
    var rig = CapsuleOnlyRecoveryRig.WithNoBankPreparedRecord();
    var verified = rig.VerifyCapsule(RecoveryCapsuleCapability.ProvedCoSerialization).Value;

    var first = rig.Coordinator.RecoverPhysical(verified);

    Assert.True(first.IsSuccess);
    Assert.True(rig.Plan.AssociatedOperationIds.AsSpan().SequenceEqual(first.OperationIds.AsSpan()));
    var committed = rig.State.Finance.TryGetVerified().Value.Journal[^1];
```

```

Assert.Equal(rig.Plan.CommandId, committed.CommandId);
Assert.Equal(rig.Plan.Timestamp, committed.Timestamp);
Assert.True(rig.Plan.PredeclaredBankDrafts[0].Parties.AsSpan().SequenceEqual(committed.Parties.AsSpan()));
Assert.True(rig.HasPermanentRequestBarrier(rig.Plan.RequestKey, rig.Plan.PayloadDigest));

var restored = rig.SaveAndRestore();
var duplicate = restored.Coordinator.RecoverPhysical(restored.VerifySameCapsule().Value);

Assert.True(first.OperationIds.AsSpan().SequenceEqual(duplicate.OperationIds.AsSpan()));
Assert.Single(restored.State.Finance.TryGetVerified().Value.Journal);
Assert.Equal(0, restored.TimestampCaptureCount);
Assert.Equal(0, restored.IdentifierIssueCount);
}

```

The evidence fixture carries the adapter's `RecoveryCapsuleCapability`; a cryptographically valid capsule is not sufficient for capsule-only completion while that capability is `UnprovedCoSerialization`. Add the withdrawal-capacity case: a valid Bank-only withdrawal whose exact output no longer fits returns `WaitForOriginalDeliveryCapacity`, retaining the same `SettlementId` and never proposing another debit. Add a coordinator-integrated contract case in which one physical mutation reserves its settlement and operation IDs, emits the journal draft plus `PhysicalSettlementControlChange`, and proves the persisted record header, settlement record, and capsule `AssociatedOperationIds` contain the same operation ID. Mutate each association independently: an unreserved settlement ID, reordered/extra/missing operation ID, payload/control settlement mismatch, or a second consumer of either ID must return `IdentifierContractViolation` without publishing finance or settlement state.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the conservative reconciliation decision table is not implemented.

□ Step 6: Implement a pure decision table with no inventory heuristics

```

public static class SettlementReconciler
{
    public static SettlementRecoveryDecision Decide(SettlementRecoveryEvidence evidence);
}

public interface IPlayerRecoveryCapsuleStore
{
    RecoveryCapsuleCapability Capability { get; }
    BankResult<SettlementRecoveryCapsule?> Read(PlayerId playerId, SettlementId settlementId);
    BankResult<bool> Stage(PlayerId playerId, SettlementRecoveryCapsule capsule);
    BankResult<bool> Remove(PlayerId playerId, SettlementId settlementId);
}

public static class SettlementCapsuleAuthenticator
{
    public static BankResult<VerifiedPhysicalRecoveryContext> VerifyAndOpen(
        RecoveryAuthenticationKey key,
        SettlementRecoveryCapsule capsule,
        EvidenceDigest expectedWorldBinding,
        PlayerId expectedPlayer,
        RecoveryCapsuleCapability capability);
}

public sealed class BankingCoordinator
{
    public CoordinatorResponse RecoverPhysical(VerifiedPhysicalRecoveryContext context);
}

```

The decision table consumes authenticated evidence flags; it never examines fungible stack totals. An unavailable capsule store reports `UnprovedCoSerialization`, which forces ambiguous one-sided evidence to quarantine. It must not masquerade as a valid absent capsule.

`VerifiedPhysicalRecoveryContext` has no public constructor and exposes no mutable plan. `VerifyAndOpen` returns it only after HMAC, canonical-plan validation, expected world/player binding, and exact `ProvedCoSerialization` capability all pass. `RecoverPhysical` is a closed coordinator path, not a general mutation hook: it accepts only physical draft types from that context, uses the plan's original command ID/timestamp/digest/parties/payload/postings/operation IDs, and assigns only current sequence/hash/revision fields. It never opens an operation scope and never captures new time or identifiers.

For capsule-only evidence, the coordinator checks that neither a conflicting settlement nor permanent request entry exists, appends the predeclared batch, installs the matching settlement tombstone, and calls an internal `RequestRegistry.FinalizeRecoveredSuccess` that can install the exact original `RequestKey`, payload digest, terminal body, operation IDs, and actual resulting revision without an active scope. That internal method is callable only from this verified recovery path. An identical existing barrier/tombstone returns its stored result; any digest, plan, response, or operation-ID disagreement quarantines the settlement. The end-to-end test starts with no Bank-side `Prepared` record and retries after another save/restart, proving the capsule contains reconstructive data rather than only a digest.

`PhysicalSettlementControlChange` is the first production `IBankControlChange`. It replaces only the immutable settlement map, declares `BankMutationDomains.Settlements`, carries the complete recovery plan and the same caller intent as its physical journal draft, and is rejected if any other state domain differs. Tests prove it can join a finance/request revision atomically without gaining a path to mutate journal, projections, replay registries, or revisions directly.

This task extends `BankState` with `BankDomainSlot<PhysicalSettlementDomainState> Settlements` and adds the gated `BankMutationReadOnly.ReadSettlements()` accessor. A corrupt settlement section therefore has no fabricated empty map and blocks all player banking as required, while verified unrelated read-only state remains available.

Extend the Task 9 tracking scope validation without changing draft IDs: each accepted physical settlement consumes exactly one ID returned by `ReserveSettlementId`, and its control record identifies the exact physical draft set for that settlement. The capsule and control record's `AssociatedOperationIds` must equal, in canonical draft order, the operation IDs that the coordinator copies into the finalized journal headers. No later phase may regenerate, substitute, or infer these IDs from array position.

`SettlementTestData`, `Evidence`, and `TestKeys` provide deterministic bounded fixtures only. The production key generator is never replaced or seeded outside tests.

□ Step 7: Record the public-API gate result honestly

Update the threat model with:

Gate result: the 1.22.7 public API exposes `IServerPlayer.SetModData/GetModData` and world-save blobs, but exposes no contract proving that player mod data and inventory are durably co-serialized or ordered against the Bank world blob. The shipping capability therefore defaults to `UnprovedCoSerialization`. A controlled engine crash matrix may promote the adapter only after repeatable evidence; until then, every ambiguous one-sided cross-blob state quarantines as specified.

This is a supported conservative outcome under section 10.1, not a claim of disk atomicity. The threat model must also record that `PhysicalSettlementRecoveryPlanV1`, not its digest, is the reconstructive authority; that sequence/hash/revision are assigned only when the plan is committed; and that the inactive-scope recovery exception is available solely through an HMAC-verified, world/player-bound capsule and the closed `RecoverPhysical` coordinator path.

Regenerate the threat model's same-basename LaTeX and PDF from Markdown, render every PDF page, and verify the revised gate text before staging the `.md` and `.tex`. Do not edit either generated format directly.

□ Step 8: Verify the recovery matrix and commit

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
git diff --check
git add FirstGearBank.Core/Settlement FirstGearBank.Core/State `
  FirstGearBank.Core/Requests/RequestRegistry.cs `
  FirstGearBank.Core/Coordination/BankingCoordinator.cs `
  tests/FirstGearBank.Core.Tests/Settlement `
  tests/FirstGearBank.Core.Tests/Coordination/BankingCoordinatorTests.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/SettlementTestData.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/Evidence.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/TestKeys.cs `
  docs/architecture/0001-physical-settlement-recovery-threat-model.md `
  docs/architecture/0001-physical-settlement-recovery-threat-model.tex
git commit -m "feat: define authenticated settlement recovery evidence"
```

Task 11: Persist Independently Checksummed Sections with Migrations and Quarantine

Files:

- Create: `FirstGearBank.Core/Persistence/BankSectionKind.cs`
- Create: `FirstGearBank.Core/Persistence/BankSectionDescriptor.cs`
- Create: `FirstGearBank.Core/Persistence/BankEnvelopeManifest.cs`
- Create: `FirstGearBank.Core/Persistence/DecodedBankEnvelope.cs`
- Create: `FirstGearBank.Core/Persistence/QuarantinedSection.cs`

- Create: `FirstGearBank.Core/Persistence/SectionQuarantineReason.cs`
- Create: `FirstGearBank.Core/Persistence/BankEnvelopeCodec.cs`
- Create: `FirstGearBank.Core/Persistence/BankPersistenceLimits.cs`
- Create: `FirstGearBank.Core/Persistence/CanonicalJson.cs`
- Create: `FirstGearBank.Core/Persistence/ISectionCodec.cs`
- Create: `FirstGearBank.Core/Persistence/ISectionMigration.cs`
- Create: `FirstGearBank.Core/Persistence/SectionMigrationRegistry.cs`
- Create: `FirstGearBank.Core/Persistence/BankStateSerializer.cs`
- Create: `FirstGearBank.Core/Persistence/BankStateLoader.cs`
- Create: `FirstGearBank.Core/Persistence/BankLoadOutcome.cs`
- Create: `FirstGearBank.Core/Persistence/InstallMarker.cs`
- Create: `FirstGearBank.Core/Persistence/InstallMarkerCodec.cs`
- Create: `FirstGearBank.Core/Persistence/IBankWorldStateStore.cs`
- Create: `FirstGearBank.Core/Persistence/BankPersistenceService.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/WorldAuthoritySectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/JournalSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/ProjectionCacheSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/RequestControlSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/ConfigurationSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/RecipientRegistrySectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/DeliveryControlSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/PhysicalSettlementSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/BranchControlSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/DiagnosticsCacheSectionV1.cs`
- Create: `FirstGearBank.Core/Persistence/Sections/RecoveryControlSectionV1.cs`
- Create: `tests/FirstGearBank.Core.Tests/Persistence/BankEnvelopeCodecTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Persistence/BankStateRoundTripTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Persistence/SectionIsolationTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Persistence/SectionMigrationRegistryTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Persistence/InstallMarkerTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Persistence/BankPersistenceServiceTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/PersistenceTestData.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/RecordingTestMigration.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/DeterministicEntropySource.cs`
- Create: `tests/FirstGearBank.Core.Tests/Fixtures/Persistence/schema-1-empty.hex`
- Create: `tests/FirstGearBank.Core.Tests/Fixtures/Persistence/schema-1-one-deposit.hex`
- Modify: `FirstGearBank.Core/State/BankState.cs:1`
- Modify: `FirstGearBank.Core/State/BankStateSnapshot.cs:1`
- Modify: `FirstGearBank.Core/State/BankStateRepository.cs:1`
- Modify: `FirstGearBank.Core/State/BankStateValidator.cs:1`
- Modify: `FirstGearBank.Core/Coordination/BankingCoordinator.cs:1`
- Modify: `tests/FirstGearBank.Core.Tests/Recovery/QuarantineTransitionTests.cs:1`
- Modify: `docs/architecture/0004-persistence-sections-and-revisions.md:1`
- Modify: `docs/architecture/0004-persistence-sections-and-revisions.tex:1` through regeneration
- Generate but do not commit: `docs/architecture/0004-persistence-sections-and-revisions.pdf`

Interfaces:

- Consumes: immutable state, replay, request control, configuration, settlement evidence, and revision contracts.
- Produces: `BankPersistenceService.Load/Save`, canonical section codecs, migration registry, raw quarantine evidence, and the `IBankWorldStateStore` port implemented by Task 12.

□ Step 1: Write failing deterministic frame and corruption tests

[Fact]

```
public void Same_sections_encode_to_the_same_bytes()
{
    var envelope = PersistenceTestData.FoundationEnvelope;

    var first = BankEnvelopeCodec.Encode(envelope).Value;
    var second = BankEnvelopeCodec.Encode(envelope).Value;

    Assert.True(first.Span.SequenceEqual(second.Span));
}
```

```

}

[Fact]
public void Corrupt_registry_payload_does_not_invalidate_verified_journal_payload()
{
    var bytes = BankEnvelopeCodec.Encode(PersistenceTestData.FoundationEnvelope).Value.ToArray();
    PersistenceTestData.FlipPayloadByte(bytes, BankSectionKind.RecipientRegistry);

    var result = BankEnvelopeCodec.Decode(bytes);

    Assert.True(result.VerifiedSections.ContainsKey(BankSectionKind.Journal));
    Assert.Equal(
        QuarantineDomain.RecipientRegistry,
        Assert.Single(result.QuarantinedSections).Domain);
}

[Fact]
public void Unknown_raw_v1_kind_quarantines_the_whole_envelope_without_reserializing_it()
{
    var stored = PersistenceTestData.EnvelopeWithUnknownRawKind(0x7fff);

    var result = BankEnvelopeCodec.Decode(stored);

    Assert.True(result.IsWholeEnvelopeQuarantined);
    Assert.True(stored.AsSpan().SequenceEqual(result.RawEnvelopeEvidence.Span));
    Assert.Empty(result.VerifiedSections);
    Assert.False(result.CanReserialize);
}

```

Add magic, container version, global revision, descriptor checksum, sort order, duplicate kind, overlap, gap, offset/length overflow, truncation, payload checksum, trailing bytes, section count, complete-size, and per-section-size tests. Run the unknown-kind case once with a checksum-valid payload and once with a checksum-invalid payload; both retain the byte-identical complete envelope and expose no partially decoded state to a reserializer.

□ Step 2: Run codec tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the envelope codec and section catalog do not exist.

□ Step 3: Implement the binary container and per-section integrity rules

Use this exact big-endian container layout:

```

4 bytes  ASCII magic "FGBK"
2 bytes  unsigned container version = 1
8 bytes  signed global revision
2 bytes  unsigned section count
4 bytes  unsigned descriptor-table byte length
N bytes  descriptor table, entries sorted by numeric section kind
32 bytes SHA-256 of descriptor table
M bytes  contiguous section payloads in descriptor order

```

descriptor entry:

```

2 bytes  unsigned raw section-kind code
2 bytes  unsigned section schema version
8 bytes  signed subsystem revision
8 bytes  unsigned payload offset relative to first payload byte
4 bytes  unsigned payload length
32 bytes SHA-256 of exact payload bytes

```

Require offsets to be contiguous and nonoverlapping, kinds unique/increasing, revisions nonnegative, and no trailing bytes. An invalid magic/header/descriptor checksum/layout causes whole-envelope quarantine. If the descriptor table is trusted, an invalid payload checksum quarantines only the section's code-owned domain and preserves its exact raw bytes.

Use these stable kinds:

```

public enum BankSectionKind : ushort
{
    WorldAuthority = 1,
    Journal = 2,
    ProjectionCache = 3,
    RequestControl = 4,
}

```

```

    Configuration = 5,
    RecipientRegistry = 6,
    DeliveryControl = 7,
    PhysicalSettlement = 8,
    BranchControl = 9,
    DiagnosticsCache = 10,
    FinancialClock = 11,
    FinancialMarket = 12,
    RecoveryControl = 13
}

```

The financial-clock and market slots are reserved now and become required only through explicit migrations in the financial-core plan. `RecoveryControl` is required in the foundation schema and stores authoritative quarantine/audit and resolved-evidence archival state; corruption in it blocks all mutation because recovery status can no longer be trusted. `BankSectionDescriptor` retains the raw `ushort` kind code before attempting enum conversion. Container version 1 has no trustworthy criticality bit, so a genuinely unknown kind causes whole-envelope read-only quarantine and exact raw envelope retention; it is never silently treated as an optional section. A later container may add an authenticated criticality rule only through an explicit migration. A known kind with an unsupported version follows the dependency matrix below and is never discarded or resaved as if understood.

```

public sealed record QuarantinedSection(
    ushort RawKind,
    ushort SchemaVersion,
    long SubsystemRevision,
    ImmutableArray<byte> DeclaredSha256,
    ImmutableArray<byte> ExactPayload,
    QuarantineDomain Domain,
    SectionQuarantineReason Reason);

```

The descriptor offset may be recomputed when healthy sections change, but every other field above and the corrupt payload bytes pass through unchanged. Re-encoding therefore preserves the failed declared hash rather than blessing corrupt bytes with a new checksum.

Use this foundation dependency matrix; later schema plans may only make it more specific with evidence:

Section problem	Services blocked	Services still permitted
<code>WorldAuthority</code> , <code>RecoveryControl</code> , invalid container, descriptor table, or unknown kind	Entire Bank; preserve/export only	Permission-gated diagnostics/export
<code>Journal</code>	All finance and automatic finance mutations	Diagnostics/export; branch repair only when independent
<code>ProjectionCache</code>	None after verified replay repair	All services allowed by other sections
<code>RequestControl</code>	Every authenticated and automatic mutation	Read-only account/history and diagnostics
<code>Configuration</code>	Config activation/edit; retain last verified active state	Services using independently verified active market state
<code>RecipientRegistry</code>	Name resolution, transfers, name-based corrections	Own-account services and unrelated automatic work
<code>DeliveryControl</code>	Transfers/corrections that require durable notices	Own deposits, withdrawals, CDs, statements, diagnostics
<code>PhysicalSettlement</code>	All player banking and finance mutation until affected players can be proved	Read-only views and diagnostics
<code>BranchControl</code>	Banker sessions and branch operations	Permission-gated nonbranch diagnostics/export
<code>DiagnosticsCache</code>	None; rebuild from verified authority	All services allowed by other sections
<code>FinancialClock</code> OR <code>FinancialMarket</code> once required	Finance mutation and quotes	Read-only views, branch repair, diagnostics

□ Step 4: Set and test structural limits

```

public static class BankPersistenceLimits
{
    public const int MaximumContainerBytes = 256 * 1024 * 1024;
}

```

```

public const ushort MaximumSections = 64;
public const int MaximumSectionBytes = 192 * 1024 * 1024;
public const int MaximumJournalRecords = 5_000_000;
public const int MaximumPostingsPerRecord = 16;
public const int MaximumPartiesPerRecord = 8;
public const int MaximumOperationIdsPerSettlement = 16;
public const int MaximumInventorySlotDeltasPerSettlement = PhysicalSettlementLimits.MaximumInventorySlotDeltas;
public const int MaximumCanonicalStackBytes = PhysicalSettlementLimits.MaximumCanonicalStackBytes;
public const int MaximumRecoveryPlanBytes = PhysicalSettlementLimits.MaximumRecoveryPlanBytes;
public const int MaximumArchivedRecoveryEvidenceBytes = RecoveryLimits.MaximumArchivedEvidenceBytes;
public const int MaximumUtf8NameBytes = 256;
public const int MaximumUtf8ReasonBytes = 4_096;
}

```

Reject sizes before allocating buffers or enumerating contents. Limits are structural safety invariants, not user config.

□ Step 5: Write failing canonical section and replay-repair tests

[Fact]

```

public void Bad_projection_cache_is_rebuilt_from_verified_journal()
{
    var stored = PersistenceTestData.StateWithValidJournalAndWrongProjectionCache;

    var outcome = BankStateLoader.Load(stored, PersistenceTestData.ExpectedWorldBinding);

    Assert.False(outcome.Quarantine.IsBlocked(QuarantineDomain.Finance));
    Assert.True(outcome.RequiresResave);
    var finance = outcome.State.Finance.TryGetVerified().Value;
    var replayed = JournalProjector.Foundation.Replay(
        finance.Journal,
        ProjectionCheckpoint.Empty).Value;
    var match = ProjectionValidator.MatchesReplay(finance.Projection, replayed);

    Assert.True(match.IsSuccess);
    Assert.True(match.Value);
}

```

[Fact]

```

public void Corrupt_journal_preserves_raw_bytes_and_never_uses_cache_as_authority()
{
    var stored = PersistenceTestData.EnvelopeWithCorruptJournalAndValidCache;

    var outcome = BankStateLoader.Load(stored, PersistenceTestData.ExpectedWorldBinding);

    Assert.True(outcome.Quarantine.IsBlocked(QuarantineDomain.Finance));
    Assert.True(stored.AsSpan().SequenceEqual(outcome.RawEnvelopeEvidence.Span));
    Assert.False(outcome.CanMutateFinance);
    Assert.True(outcome.State.Finance.IsUnavailable);
    Assert.False(outcome.State.Finance.TryGetVerified().IsSuccess);
}

```

[Fact]

```

public void Corrupt_journal_does_not_require_a_fake_empty_finance_state_for_an_independent_config_commit()
{
    var rig = PersistenceRecoveryRig.Load(PersistenceTestData.EnvelopeWithCorruptJournal);
    var originalEvidence = rig.RawSection(BankSectionKind.Journal).ToArray();

    var response = rig.Coordinator.Execute(
        RequestTestData.Context(sequence: 1),
        new ConfigurationOnlyMutation(ConfigurationTestData.ValidDisplayPrecisionFour));
    rig.StageAndReload();

    Assert.True(response.IsSuccess);
    Assert.True(rig.State.Finance.IsUnavailable);
    Assert.Equal(4, rig.State.Configuration.TryGetVerified().Value.DisplayPrecision);
    Assert.True(originalEvidence.AsSpan().SequenceEqual(rig.RawSection(BankSectionKind.Journal).Span));
}

```

Add roundtrips for every foundation journal payload and invocation variant, request high-water/cache/permanent request key, permanent automatic key/digest/result, configuration candidate, empty current registry epoch, empty delivery/branch state, all settlement phases/complete recovery plans/capsules, recovery-control evidence/audit, every verified/unavailable domain-slot variant, revision vector, and raw diagnostics. A loader may never satisfy a missing authoritative slot with an empty object. Add the decisive pass-through scenario: load a corrupt registry alongside a valid journal,

commit a healthy permitted financial revision, stage and reload the envelope, then prove the journal advanced while the registry remains quarantined with byte-identical corrupt payload, declared hash, schema version, raw kind, and subsystem revision.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because section codecs, replay repair, and complete state reconciliation are not implemented.

□ Step 6: Implement explicit DTO codecs and state reconciliation

Each section codec maps domain state to a versioned DTO; domain interfaces are never passed directly to `JsonSerializer`. Canonical JSON uses declaration-ordered properties, UTF-8 without BOM, no indentation, invariant integer numbers, `G29` strings for decimals, `R` strings for finite doubles, lowercase hex IDs/hashes, and sorted arrays instead of dictionaries. Deserialize with strict duplicate-property, enum, size, and trailing-token checks.

`WorldAuthoritySectionV1` contains the `WorldInstanceId`, the SHA-256 world binding, and the 32-byte recovery authentication key. Its canonical binding is `SHA-256("FGB/WORLD-BINDING/1\0" || UInt32BigEndian(identifierLength) || UTF8(SavegameIdentifier))`. Corruption or mismatch in this section quarantines the complete envelope because no other section may be imported into a different world.

This task extends `BankStateSnapshot` and its repository-owned immutable publication unit with a bounded `ImmutableArray<QuarantinedSection>` opaque-section set. Coordinator candidates carry the same set by reference and cannot read, remove, or rewrite it; only the loader and the typed recovery path may create or consume an entry. Snapshot capture therefore cannot lose pass-through evidence while a healthy domain publishes concurrently. `BankingCoordinator.ExecuteRecovery` receives an internal repository capability that can select only the one opaque entry whose domain and `EvidenceDigest` match its `QuarantineTransition`; ordinary `BankMutationReadOnlyView` never receives that capability. A successful clear moves those exact bytes into `RecoveryControlState` and removes the opaque entry in the same single snapshot publication. Add a test that changes the opaque evidence between authorization and commit and prove the stale recovery rejects without replacing the unavailable slot.

`BankStateLoader` validates the world binding and authority key, decodes/migrates each section, verifies the journal, replays from a verified checkpoint or sequence one, compares the cache, repairs only derivable caches, clears active operation scopes for restart, retains permanent request and automatic-operation barriers, and assembles the quarantine state before returning any service access. Every authoritative domain becomes either `BankDomainSlot.Verified(value)` or `BankDomainSlot.Unavailable(domain, evidenceDigest)`; the loader never substitutes an empty value for unreadable authority. `RecoveryControlSectionV1` is required, structurally bounded, and cross-checks every unavailable slot/quarantine/archive entry. `RequestControlSectionV1` owns both registries so no finance path can load without the exact deduplication state required for its invocation keys. Startup cross-checks every journal request/automatic invocation against the corresponding permanent entry, digest, operation-ID set, and revision; a missing or contradictory barrier is authoritative corruption, not a cache repair. `BankStateSerializer` receives both healthy domain state and retained `QuarantinedSection` values. It emits quarantined known sections as opaque pass-through payloads with their original declared hashes and meta-data while healthy section descriptors and the table checksum advance normally. It serializes no payload for an unavailable slot except that slot's exact opaque pass-through section. Once an authorized repair clears a slot, the old opaque bytes must already exist byte-identically in `RecoveryControlState.EvidenceArchive`; only then may the repaired section replace the pass-through entry. Whole-envelope quarantine never enters this reserializer.

Fresh-state creation requests its world ID and 32-byte recovery key from the Task 5 `IEntropySource`. Task 12 wires `CryptographicEntropySource`; golden fixtures inject only the listed deterministic test implementation.

□ Step 7: Write failing migration and unknown-version tests

```
[Fact]
public void Migration_validates_old_and_new_sections_before_replacement()
{
    var migration = new RecordingTestMigration(fromVersion: 1, toVersion: 2);
    var registry = SectionMigrationRegistry.Create([migration]);

    var result = registry.Migrate(
        BankSectionKind.DiagnosticsCache,
        sourceVersion: 1,
        PersistenceTestData.ValidDiagnosticsV1);

    Assert.True(result.IsSuccess);
    Assert.Equal(["validate-old", "transform", "validate-new"], migration.Calls);
}
```

Test missing link, duplicate edge, cycle, nondeterministic output detection, invalid old/new data, unknown newer container, unknown newer critical section, and preservation of its raw bytes.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because deterministic migration-chain validation is not implemented.

□ Step 8: Implement deterministic per-section migration chains

```
public interface ISectionMigration
{
    BankSectionKind SectionKind { get; }
    ushort FromVersion { get; }
    ushort ToVersion { get; }

    BankResult<ReadOnlyMemory<byte>> Migrate(ReadOnlyMemory<byte> verifiedSource);
}
```

The registry accepts only $n \rightarrow n+1$ edges, one edge per (kind, from), acyclic complete paths, and byte-identical output when the same migration is invoked twice in tests. Source validation happens before transformation and target validation before candidate installation. No migration writes live state in place.

□ Step 9: Write failing install-marker and world-save-staging tests

Use two namespaced world keys so first installation can be distinguished from deleted authority:

```
firstgearbank:installed
firstgearbank:world-state
```

Test this matrix:

```
marker absent + state absent -> create fresh unstaged state; block mutations until initial snapshot is staged
marker absent + state valid -> load state and repair marker on the next world-save staging callback
marker present + state absent -> whole-Bank quarantine; do not create defaults
marker/state binding mismatch -> whole-Bank quarantine
marker present + state valid -> normal load
```

Also test store exception, serialization exception, exact-revision unstaged-flag clearing, a newer publish during staging, the always-on engine-restage flag, and that a first-save marker-stage failure after a successful envelope stage reloads the verified state and repairs only the marker.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL because the install marker and world-save staging protocol are not implemented.

□ Step 10: Implement the persistence service and two-key protocol

```
public interface IBankWorldStateStore
{
    ReadOnlyMemory<byte>? ReadInstallMarker();
    ReadOnlyMemory<byte>? ReadEnvelope();
    void StageInstallMarker(ReadOnlyMemory<byte> bytes);
    void StageEnvelope(ReadOnlyMemory<byte> bytes);
}

public sealed class BankPersistenceService
{
    public BankLoadOutcome Load(string savegameIdentifier);
    public BankResult<BankRevision> StageForWorldSave(BankStateRepository repository);
}
```

On save, serialize an immutable snapshot first, then stage the envelope before the install marker. On first installation, the marker is the final commit flag; a crash after the envelope stage is therefore the recoverable **marker absent + state valid** case instead of the unrecoverable **marker present + state absent** case. The marker contains container version, WorldInstanceId, and world-binding hash but never the recovery HMAC key. If a crash leaves marker-only state, the next load quarantines safely. If state exists without a marker, its verified embedded world authority permits marker repair. **HasUnstagedChanges** clears only after both staging calls return for the exact captured revision; **RequiresEngineSaveRestage** does not. This result means accepted by the engine's pre-save world-data staging API, not written to disk. The service calls **MarkStagingSucceeded/MarkStagingFailed**; the host restages on every later **GameWorldSave** because 1.22.7 exposes no post-save acknowledgment.

□ Step 11: Add stable golden fixtures

Encode one empty state and one single-deposit state using a deterministic test entropy source, write uppercase byte-pair hex with one line per 32 bytes, then make tests decode and compare against freshly encoded bytes. Fixture regeneration must be an explicit test utility argument and ordinary test runs must fail on a byte change.

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: PASS for roundtrip, deterministic bytes, migration, cache repair, isolation, limits, and staging behavior.

□ Step 12: Commit the persistence schema and decision record

Regenerate the decision record's same-basename .tex and .pdf from Markdown and visually verify the rendered PDF before staging only the Markdown and LaTeX sources.

```
git diff --check
git add FirstGearBank.Core/Persistence FirstGearBank.Core/State `
  FirstGearBank.Core/Coordination/BankingCoordinator.cs `
  tests/FirstGearBank.Core.Tests/Persistence `
  tests/FirstGearBank.Core.Tests/Recovery/QuarantineTransitionTests.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/PersistenceTestData.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/RecordingTestMigration.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/DeterministicEntropySource.cs `
  tests/FirstGearBank.Core.Tests/Fixtures/Persistence `
  docs/architecture/0004-persistence-sections-and-revisions.md `
  docs/architecture/0004-persistence-sections-and-revisions.tex
git commit -m "feat: add sectioned bank state persistence"
```

Task 12: Wire the Thin Vintage Story Server Lifecycle, Configuration, Capsule, and Log Adapters

Files:

- Create: First Gear Bank/FirstGearBankModSystem.cs
- Create: First Gear Bank/Runtime/ServerHostState.cs
- Create: First Gear Bank/Runtime/FirstGearBankServerHost.cs
- Create: First Gear Bank/Persistence/VintageStoryWorldStateStore.cs
- Create: First Gear Bank/Configuration/VintageStoryConfigurationStore.cs
- Create: First Gear Bank/Settlement/VintageStoryPlayerCapsuleStore.cs
- Create: First Gear Bank/Diagnostics/FgbFileLogger.cs
- Create: First Gear Bank/Diagnostics/VintageStoryBankLogger.cs
- Delete: First Gear Bank/First Gear BankModSystem.cs
- Create: tests/FirstGearBank.Adapter.Tests/Runtime/FirstGearBankServerHostTests.cs
- Create: tests/FirstGearBank.Adapter.Tests/Configuration/VintageStoryConfigurationStoreTests.cs
- Create: tests/FirstGearBank.Adapter.Tests/Diagnostics/FgbFileLoggerTests.cs
- Create: tests/FirstGearBank.Adapter.Tests/Persistence/WorldStateStoreContractTests.cs
- Create: tests/FirstGearBank.Adapter.Tests/Settlement/PlayerCapsuleStoreContractTests.cs
- Create: tests/FirstGearBank.Adapter.Tests/TestSupport/FakeBankWorldStateStore.cs
- Create: tests/FirstGearBank.Adapter.Tests/TestSupport/ServerHostTestRig.cs
- Create: tests/FirstGearBank.Adapter.Tests/TestSupport/TemporaryDirectory.cs
- Create: tests/FirstGearBank.Adapter.Tests/TestSupport/TestClock.cs
- Create: tests/FirstGearBank.Adapter.Tests/TestSupport/TestTemplate.cs
- Create: tests/FirstGearBank.Adapter.Tests/TestSupport/LogTestReader.cs
- Modify: First Gear Bank/First Gear Bank.csproj:1
- Modify: README.md:1
- Modify: docs/testing/foundation-engine-smoke.md:1
- Modify: docs/testing/foundation-engine-smoke.tex:1 through regeneration
- Generate but do not commit: docs/testing/foundation-engine-smoke.pdf

Interfaces:

- Consumes: IBankWorldStateStore, BankPersistenceService, IPlayerRecoveryCapsuleStore, configuration parser, and IBankLog from the core.
- Produces: the only direct Vintage Story lifecycle/storage adapters and a foundation mod that loads under 1.22.7.

□ Step 1: Write failing host lifecycle tests behind fake stores

```
[Fact]
public void Fresh_world_stays_mutation_blocked_until_its_initial_state_is_staged()
{
    var store = new FakeBankWorldStateStore(marker: null, envelope: null);
    using var host = ServerHostTestRig.Create(store);

    host.OnSaveGameLoaded();

    Assert.Equal(ServerHostState.AwaitingInitialSave, host.State);
    Assert.False(host.CanAcceptMutations);

    host.OnGameWorldSave();

    Assert.Equal(ServerHostState.Ready, host.State);
    Assert.True(host.CanAcceptMutations);
    Assert.NotNull(store.Marker);
    Assert.NotNull(store.Envelope);
}

[Fact]
public void Repeated_load_callbacks_do_not_replace_live_state()
{
    var store = FakeBankWorldStateStore.WithValidEnvelope();
    using var host = ServerHostTestRig.Create(store);

    host.OnSaveGameLoaded();
    var first = host.Repository;
    host.OnSaveGameLoaded();

    Assert.Same(first, host.Repository);
    Assert.Equal(1, store.EnvelopeReadCount);
}
```

Add valid restore, marker-only quarantine, corrupt finance, isolated registry corruption, staging exception, newer revision during staging, repeated restaging without a new revision, create/load event order, save callback before load, and disposed-host callback tests.

❑ Step 2: Run host tests and observe missing-type failures

```
dotnet test tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj -c Release
```

Expected: FAIL because the host does not exist.

❑ Step 3: Implement a testable host with explicit lifecycle states

```
public sealed class FirstGearBankServerHost : IDisposable
{
    public ServerHostState State { get; }
    public bool CanAcceptMutations { get; }
    public BankStateRepository? Repository { get; }

    public void OnSaveGameCreated();
    public void OnSaveGameLoaded();
    public void OnGameWorldSave();
    public void Dispose();
}
```

The host is constructed with the savegame identifier, world-state store, configuration store, entropy source, section codecs/migrations, and logger. The first create/load callback performs exactly one load. Fresh state is view-only until the first successful staging of marker and envelope. Quarantine enables only domains allowed by the load outcome. `OnGameWorldSave` captures one immutable revision and calls `StageForWorldSave` even when no newer revision exists. It logs "staged for engine world save" or the exact staging failure, never "durably saved". The host cannot clear `RequiresEngineSaveRestage` because the public event fires before disk writing and exposes no completion callback.

The listed test-support types implement only project-owned ports, fixed clocks/templates, and scoped temporary directories. They must not mock or reimplement Vintage Story behavior.

❑ Step 4: Verify host tests pass

```
dotnet test tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj -c Release
```

❑ Step 5: Write failing file-log and configuration-file tests

```
[Fact]
public void File_log_uses_only_the_four_exact_labels()
{
    using var directory = TemporaryDirectory.Create();
    using var log = new FgbFileLogger(directory.Path, maximumBytes: 2048, retainedFiles: 2, TestClock.UtcNow);

    log.Write(BankLogLevel.Info, "Host", "started");
    log.Write(BankLogLevel.Debug, "Host", "detail");
    log.Write(BankLogLevel.Warning, "Host", "degraded");
    log.Write(BankLogLevel.Critical, "Host", "blocked");

    Assert.Equal(
        ["INFO", "DEBUG", "WARN", "CRIT"],
        LogTestReader.ReadLabels(
            Path.Combine(directory.Path, "logs", "firstgearbank.log")));
}

[Fact]
public void Valid_existing_configuration_is_read_without_rewrite()
{
    using var directory = TemporaryDirectory.Create();
    var path = directory.Write("firstgearbank.jsonc", "{\n\"TransferCooldownSeconds\":2.0}");
    var before = File.GetLastWriteTimeUtc(path);

    var result = new VintageStoryConfigurationStore(path, TestTemplate.Text).Load();

    Assert.Equal(2.0, result.Candidate.TransferCooldownSeconds);
    Assert.Equal(before, File.GetLastWriteTimeUtc(path));
}
```

Add missing-config canonical creation, invalid-config no rewrite, explicit owned scalar save, one-time legacy migration backup, atomic temp-file replacement, failed write preserving original, exact 2 MiB rotation threshold, five-file retention, concurrent writes, exception rendering, and no UID/balance fields in host lifecycle log calls.

```
dotnet test tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj -c Release
```

Expected: FAIL because bounded file logging and owned config-file behavior are not implemented.

□ Step 6: Implement bounded logging and comment-preserving configuration storage

Write logs beneath <runtime-data>/FirstGearBank/logs/firstgearbank.log, rotate before a write would exceed 2 MiB, retain exactly five completed files, and format each line as:

```
2026-09-08T12:34:56.7890000Z INFO Host message
```

Escape embedded CR/LF so one event remains one record. VintageStoryBankLogger writes to the file first and maps INFO -> Notification, DEBUG -> Debug, WARN -> Warning, and CRIT -> Fatal on the native logger.

Place the live config at <ModConfig>/FirstGearBank/firstgearbank.jsonc. On absence, atomically copy the embedded canonical template. Ordinary load never writes. An explicit owned edit applies JsoncSyntaxDocument spans and replaces through a same-directory temporary file. The one legacy migration writes a .pre-fgb-migration backup once before atomic replace.

□ Step 7: Verify log/config tests pass

```
dotnet test tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj -c Release
```

□ Step 8: Write and run failing engine-storage adapter contract tests

Use the internal delegate seams to prove key selection and buffer ownership without pretending to emulate engine saves:

```
[Fact]
public void World_store_clones_bytes_read_from_and_staged_to_the_engine_boundary()
{
    var engineBytes = new byte[] { 1, 2, 3 };
    byte[]? stagedBytes = null;
    var store = new VintageStoryWorldStateStore(
        key => key == VintageStoryWorldStateStore.EnvelopeKey ? engineBytes : null,
        (key, bytes) => stagedBytes = bytes);

    var read = store.ReadEnvelope().Value.ToArray();
    read[0] = 9;
}
```

```

var input = new byte[] { 4, 5, 6 };
store.StageEnvelope(input);
input[0] = 9;

Assert.Equal(1, engineBytes[0]);
Assert.Equal(4, stagedBytes![0]);
}

```

Add capsule-store cases for exact player/key routing, multiple sorted settlement IDs, stage/read/remove roundtrip, clone ownership, duplicate IDs, structural bounds, corrupt/unknown-version bytes, and the default `UnprovedCoSerialization` capability.

```
dotnet test tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj -c Release
```

Expected: FAIL because the Vintage Story storage adapters do not exist.

□ Step 9: Write the Vintage Story storage adapters with exact namespaced keys

```

public sealed class VintageStoryWorldStateStore : IBankWorldStateStore
{
    public const string InstallMarkerKey = "firstgearbank:installed";
    public const string EnvelopeKey = "firstgearbank:world-state";

    public VintageStoryWorldStateStore(ISaveGame saveGame);
    public ReadOnlyMemory<byte>? ReadInstallMarker();
    public ReadOnlyMemory<byte>? ReadEnvelope();
    public void StageInstallMarker(ReadOnlyMemory<byte> bytes);
    public void StageEnvelope(ReadOnlyMemory<byte> bytes);
}

```

Use `ISaveGame.GetData(string)` and `StoreData(string, byte[])` only. Clone input/output buffers at the adapter boundary so the engine and core never share mutable byte arrays.

`VintageStoryPlayerCapsuleStore` uses `IServerPlayer.SetModData<byte[]>` and `GetModData<byte[]>` under the key `firstgearbank:physical-settlements`, but reports capability `UnprovedCoSerialization` until the controlled crash matrix has repeatably proved a stronger result. It never exposes this storage as automatic proof merely because bytes exist.

Keep the public constructors API-backed. Add internal delegate-backed constructors, exposed only to `FirstGearBank.Adapter.Tests` through an SDK `InternalsVisibleTo` item in the adapter project file, so the storage contract tests can record exact keys and buffer cloning without implementing the large Vintage Story interfaces. Those delegates are call spies, not substitutes for engine durability tests. Store a versioned, bounded, settlement-ID-sorted capsule collection per player; malformed, duplicate, oversized, or unknown-version bytes return `InvalidSettlementEvidence` and are never overwritten during load.

□ Step 10: Replace the empty ModSystem with wiring only

```

public sealed class FirstGearBankModSystem : ModSystem
{
    private ICoreServerAPI? _serverApi;
    private FirstGearBankServerHost? _serverHost;

    public override void StartServerSide(ICoreServerAPI api);
    public override void Dispose();
}

```

`StartServerSide` validates assembly version 1.22.7.0, creates data/config/log paths with verified public APIs, constructs the adapters/host, and subscribes host methods to `SaveGameCreated`, `SaveGameLoaded`, and `GameWorldSave`. `Dispose` unsubscribes the exact delegates before disposing the host/logger and is safe if startup stopped partway. Do not initialize client authority or register packet DTOs in this phase.

□ Step 11: Compile the direct engine boundary and run adapter tests

```

dotnet build "First Gear Bank/First Gear Bank.csproj" -c Release
dotnet test tests/FirstGearBank.Adapter.Tests/FirstGearBank.Adapter.Tests.csproj -c Release

```

Expected: zero errors; test output may contain the test-only API DLL, while release package verification still excludes it.

□ Step 12: Package and perform the controlled 1.22.7 smoke run

```
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=VerifyPackage
```

Deploy explicitly to the disposable StoryForge test installation, create a backed-up disposable world, and execute the matrix in `docs/testing/foundation-engine-smoke.md`. At minimum verify: mod loads; exact game/API version is logged; fresh state stages on the first world save; restart restores the same revision and world ID; copied corruption quarantines; no client-side financial service starts; a diagnostic capsule roundtrips through an ordinary player save/restart without any inventory claim; and the file log contains one bounded startup/restore result. Mark crash ordering and every synthesized physical-settlement phase as deferred to Phase 3 rather than manufacturing evidence in a foundation-only mod.

Record the exact ZIP SHA-256, world backup, observed revisions, and pass/fail in the smoke document. A normal capsule roundtrip proves persistence only; it does not promote `UnprovedCoSerialization` or claim inventory ordering.

Regenerate the smoke document's `.tex` and `.pdf` from Markdown and visually verify the rendered evidence before commit.

□ Step 13: Commit the adapter and observed smoke result

```
git diff --check
git add "First Gear Bank/FirstGearBankModSystem.cs" "First Gear Bank/Runtime" `
"First Gear Bank/Persistence" "First Gear Bank/Configuration" "First Gear Bank/Settlement" `
"First Gear Bank/Diagnostics" "First Gear Bank/First Gear Bank.csproj" `
tests/FirstGearBank.Adapter.Tests README.md docs/testing/foundation-engine-smoke.md `
docs/testing/foundation-engine-smoke.tex
git add -u "First Gear Bank/First Gear BankModSystem.cs"
git commit -m "feat: wire Vintage Story foundation lifecycle"
```

Task 13: Prove the Foundation as One Recoverable Packaged Vertical Slice

Files:

- Create: `tests/FirstGearBank.Core.Tests/Integration/FoundationRecoveryTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/Architecture/CoreDependencyTests.cs`
- Create: `tests/FirstGearBank.Adapter.Tests/Integration/FoundationHostIntegrationTests.cs`
- Create: `tests/CakeBuild.Tests/SourceStandardsTests.cs`
- Create: `tests/FirstGearBank.Core.Tests/TestSupport/FoundationRecoveryRig.cs`
- Create: `docs/testing/foundation-acceptance.md`
- Create: `docs/testing/foundation-acceptance.tex` through regeneration
- Generate but do not commit: `docs/testing/foundation-acceptance.pdf`
- Modify: `README.md:1`
- Modify: `docs/superpowers/plans/2026-09-08-first-gear-bank-v1-roadmap.md:1`
- Modify: `docs/superpowers/plans/2026-09-08-first-gear-bank-v1-roadmap.tex:1` through regeneration
- Generate but do not commit: `docs/superpowers/plans/2026-09-08-first-gear-bank-v1-roadmap.pdf`

Interfaces:

- Consumes: every Task 1-12 deliverable.
- Produces: a green, packaged, source-linked Phase 1 baseline and recorded evidence for authoring the financial-core plan.

□ Step 1: Write the failing save/reload/replay integration test

```
[Fact]
public void Foundation_state_replays_in_process_but_rejects_the_scope_after_restart()
{
    var rig = FoundationRecoveryRig.CreateDeterministic();
    var scope = rig.OpenScope(PlayerIds.Ada);
    var request = rig.FinalizedDepositTestRequest(scope, sequence: 1, units: 1_000_000);

    var first = rig.Coordinator.Execute(request.Context, request.Mutation);
    var inProcessRetry = rig.Coordinator.Execute(request.Context, request.Mutation);

    Assert.True(first.IsSuccess);
    Assert.Equal(first.InvocationKey, inProcessRetry.InvocationKey);
    Assert.Equal(first.ResultingRevision, inProcessRetry.ResultingRevision);
    Assert.True(first.OperationIds.AsSpan().SequenceEqual(inProcessRetry.OperationIds.AsSpan()));

    var saved = rig.Persistence.EncodeCurrent();
    var restored = FoundationRecoveryRig.Restore(saved);
    var beforeStaleRetry = restored.State;
    var staleRetry = restored.Coordinator.Execute(request.Context, request.Mutation);
```

```

Assert.False(staleRetry.IsSuccess);
Assert.Equal(BankErrorCode.InactiveRequestScope, staleRetry.Failure?.Code);
Assert.Equal(beforeStaleRetry.Revisions.Global, restored.State.Revisions.Global);
Assert.Equal(0, restored.TimestampCaptureCount);
Assert.Equal(0, restored.IdentifierIssueCount);
var restoredFinance = restored.State.Finance.TryGetVerified().Value;
Assert.Single(restoredFinance.Journal);
Assert.Equal(
    PhysicalSettlementPhase.Finalized,
    Assert.Single(restored.State.Settlements.TryGetVerified().Value.Records).Phase);
var replayed = restored.Projector.Replay(
    restoredFinance.Journal,
    ProjectionCheckpoint.Empty).Value;
var projectionMatch = ProjectionValidator.MatchesReplay(restoredFinance.Projection, replayed);

Assert.True(projectionMatch.IsSuccess);
Assert.True(projectionMatch.Value);
}

```

This uses a matching finalized settlement fixture, not a claim that payable inventory mutation exists. Add an admitted rejection retry before staging, pruned response, fresh-scope success after restart, restored automatic-key duplicate, marker repair, registry-section corruption with finance available, journal corruption with finance read-only, and exact raw-byte preservation.

□ Step 2: Run the integration tests and observe the first missing fixture/behavior failure

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

Expected: FAIL until the deterministic integration rig and any exposed contract gaps are completed.

□ Step 3: Add only the deterministic fixture code needed by the integration tests

Keep fake time, entropy, identifiers, and world storage under the test project. Drive production coordinator, journal, projection, codecs, migration registry, and loader without replacing them with mocks. Use the same canonical request bytes before and after restore.

□ Step 4: Verify the full recovery test matrix passes

```
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release
```

□ Step 5: Add architecture and source-standard regression tests

CoreDependencyTests loads FirstGearBank.Core.dll and rejects any referenced assembly whose name starts with Vintagestory, Cake, or the adapter assembly name. SourceStandardsTests walks every human-maintained .cs file in the repository working tree, including files not yet committed, while excluding only bin, obj, Artifacts, and generated code. It requires the first token to be a /* ... */ responsibility block and rejects documentation lines longer than 120 characters. Do not obtain the file list from git ls-files, because that would skip the new files at the very gate intended to check them. During the same review, inspect every callable changed in Phase 1 for its immediately preceding //// block and exact three-blank-line spacing; do not use a brittle declaration regex as false assurance.

```

[Fact]
public void Core_has_no_engine_or_build_tool_dependency()
{
    var forbidden = typeof(CoreAssemblyMarker).Assembly
        .GetReferencedAssemblies()
        .Where(reference =>
            reference.Name?.StartsWith("Vintagestory", StringComparison.OrdinalIgnoreCase) == true ||
            reference.Name?.StartsWith("Cake", StringComparison.OrdinalIgnoreCase) == true ||
            reference.Name == "FirstGearBank")
        .ToArray();

    Assert.Empty(forbidden);
}

```

```

dotnet test tests/CakeBuild.Tests/CakeBuild.Tests.csproj -c Release
dotnet test tests/FirstGearBank.Core.Tests/FirstGearBank.Core.Tests.csproj -c Release

```

Expected: FAIL on any remaining source-layout or dependency-boundary violation; correct those violations before the final verification step.

□ Step 6: Run formatting, source, dependency, vulnerability, and full test checks

```
dotnet format "First Gear Bank.sln" --verify-no-changes --no-restore
dotnet test "First Gear Bank.sln" -c Release
dotnet list "First Gear Bank.sln" package --vulnerable --include-transitive
git diff --check
```

Expected: formatter reports no change; all tests pass; NuGet reports no known vulnerable package; whitespace check passes. If the formatter conflicts with the required three-blank-line standard, preserve the mandated source layout and record the specific formatter limitation instead of rewriting documented callables incorrectly.

□ Step 7: Build and inspect the final foundation archive twice

```
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=VerifyPackage
$archive = Get-Item -LiteralPath "Releases/firstgearbank_1.0.0.zip"
$firstHash = (Get-FileHash -LiteralPath $archive.FullName -Algorithm SHA256).Hash
dotnet run --project CakeBuild/CakeBuild.csproj -- --target=VerifyPackage
$secondHash = (Get-FileHash -LiteralPath $archive.FullName -Algorithm SHA256).Hash
if ($firstHash -ne $secondHash) { throw "Verified package bytes changed between identical builds." }
```

Expected package root: modinfo.json, FirstGearBank.dll, FirstGearBank.Core.dll, and the configuration template/resource or assets intentionally emitted by the adapter. Expected absence: VintagestoryAPI.dll, VSSurvivalMod.dll, Cake DLLs, test assemblies, PDBs, .deps.json, and .runtimeconfig.json.

□ Step 8: Complete the acceptance record from command and engine evidence

docs/testing/foundation-acceptance.md must record exact command lines, exit codes, test counts, ZIP hash/layout, installed API assembly version, fresh/save/restart smoke results, quarantine smoke results, log path/labels, unresolved engine gates, and the accrued-liability-index gate result. Do not mark a row passing from intended behavior or mock output alone.

Update README.md to state that the foundation is installed but player-facing banking remains unavailable until the later roadmap phases. Mark Phase 1 complete in the roadmap only when every automated check and required 1.22.7 smoke row has observed evidence. Inventory/capsule ordering and per-phase process-crash rows are explicitly deferred Phase 3 evidence, not required Phase 1 rows. Regenerate the acceptance record and roadmap .tex/.pdf files from Markdown and visually verify every page; stage only Markdown and LaTeX.

□ Step 9: Review the diff against the locked phase scope

```
git status --short
git diff --stat
git diff --check
rg -n "Vintagestory|ICore|Entity|BlockPos|ItemStack" FirstGearBank.Core
```

Expected: the final search returns only architecture-boundary test strings or explanatory exclusions, never a core engine type/reference. Confirm that no loan, overdraft, margin, repo, swap, stock, fund, option, commodity, joint account, cross-world migration, currency conversion, temporal CD, CD redemption, or transfer fee behavior entered the diff.

□ Step 10: Commit the verified foundation gate

```
git add tests/FirstGearBank.Core.Tests/Integration tests/FirstGearBank.Core.Tests/Architecture `
  tests/FirstGearBank.Adapter.Tests/Integration tests/CakeBuild.Tests/SourceStandardsTests.cs `
  tests/FirstGearBank.Core.Tests/TestSupport/FoundationRecoveryRig.cs `
  docs/testing/foundation-acceptance.md docs/testing/foundation-acceptance.tex README.md `
  docs/superpowers/plans/2026-09-08-first-gear-bank-v1-roadmap.md `
  docs/superpowers/plans/2026-09-08-first-gear-bank-v1-roadmap.tex
git commit -m "test: verify the recoverable banking foundation"
git status --short
```

Expected: clean status except for any explicitly documented user-owned pre-existing changes.

Plan Self-Review

Foundation specification coverage

- Sections 1-3: Tasks 1-4 establish exact target/mod identity, pure-core boundary, server authority, and safe packaging.
- Section 4: Task 5 implements unit scale, ties-to-even conversion, display independence, checked arithmetic, and cap.
- Section 5 foundation: Tasks 5 and 8 implement hidden world/player identities and the exclusive lazy-opening rule.
- Section 6: Tasks 7-9 implement immutable double entry, typed facts, replay, revisions, and request plus automatic-operation idempotency.

- Section 18 foundation: Task 6 implements every static field/default/domain and JSONC ownership; derived activation is explicitly gated on the financial formulas that consume it.
- Sections 20.2-20.4 foundation: Tasks 9-12 implement snapshots, staging semantics, integrity, migrations, replay, request and automatic-key persistence, explicit unavailable-domain slots, opaque quarantined-section pass-through, recovery evidence archival, and versioned hash domains.
- Section 21: every task carries failure, privacy, trust, bounded-allocation, logging, and no-scan constraints.
- Section 22: Tasks 1, 10, and 12 record verified APIs and isolate runtime-only evidence.
- Section 23 foundation rows: Tasks 3-13 exercise package safety, money, journal, replay, request, schema, corruption, settlement evidence, adapter lifecycle, and multiplayer-independent recovery behavior.
- Sections 7-17 and remaining 18-26 behavior are allocated to named executable plans in the roadmap and are not claimed by this foundation milestone.

Type and boundary consistency

- `PlayerId`, both invocation-key variants, `CommandPayloadDigest`, `FinancialTimestamp`, `BankUnits`, and the revision vector originate in Task 5 and are reused without aliases.
- `JournalRecord` is the only monetary source accepted by projection and persistence.
- `BankStateRepository` is the only published-state owner; the coordinator is the only mutation writer.
- Mutations see only a domain-gated read view; unavailable authority is never represented by a fabricated empty value.
- `IBankWorldStateStore`, `IPlayerRecoveryCapsuleStore`, `IFinancialTimestampSource`, and `IBankLog` point from core toward adapter implementations, never from core toward Vintage Story.
- Persistence separates journal authority from projection caches and separates registry, delivery, settlement, branch, configuration, clock, and market failure domains.
- A known quarantined section remains opaque and byte-preserved across permitted healthy saves; an unknown section kind blocks whole-envelope mutation because container v1 has no authenticated criticality flag.
- `GameWorldSave` is a pre-write staging boundary. No code or log calls its return a durable disk acknowledgment.
- The complete authenticated physical-recovery plan is reconstructive, while its cross-inventory co-serialization remains an engine-evidence gate; the accrued-liability result remains a separate mathematical evidence gate. Neither is represented as stronger than the API or proof supports.